

Trust Region Q Adjoint Matching

Yonghoon Dong¹

yonghoon.dong@kaist.ac.kr

Kyungmin Lee¹

kyungmnlee@kaist.ac.kr

Changyeon Kim¹

changyeon.kim@kaist.ac.kr

Jaehyuk Kim²

waewae1@snu.ac.kr

Jinwoo Shin^{1,3}

jinwoos@kaist.ac.kr

Abstract

Off-policy reinforcement learning of pretrained flow policies remains challenging due to the instability of optimization arising from the multi-step sampling process. Recently, Q-learning with Adjoint Matching (QAM) addressed this issue by reformulating into a memoryless stochastic optimal control (SOC) problem with a learned critic. However, QAM inherits a fundamental fragility of critic-guided improvement: small critic errors are amplified when critics are ill-conditioned, often leading to model collapse. This paper introduces Trust Region Q-Adjoint Matching (TRQAM), a stable off-policy fine-tuning algorithm that adaptively controls the path-space KL with pretrained flow policies through projected dual descent. Specifically, we optimize the trust-region parameter λ in SOC dynamics, and theoretically show that the path-space KL can be represented by a closed-form function of λ . As a result, our method can precisely control the exact deviation from pretrained flow policies, achieving stable off-policy RL. Through experiments on 50 OGBench tasks, TRQAM consistently outperforms prior arts in both offline RL and offline-to-online RL. In particular, TRQAM achieves an overall success rate of 68% in offline RL, substantially improves the strongest baseline at 46%.

1 Introduction

Recently, flow matching policies [2, 7, 27, 29] have emerged as a promising approach to model rich and diverse action distributions, enabling high-capacity behavior generation beyond conventional uni-modal Gaussian policies. A pretrained flow policy captures useful skills, behavioral constraints, and broad coverage over plausible actions, making it an attractive prior for downstream off-policy RL fine-tuning [4, 11, 17–20, 45, 48, 51, 57, 60].

However, as the flow policy is defined implicitly through a multi-step denoising process, gradient-based policy improvement requires differentiating through the multi-step sampling chain, making direct backpropagation expensive and unstable [39, 59]. Existing approaches sidestep this through residual-style methods that keep the pretrained policy frozen and learn an additive residual to its actions [11, 55], or noise-space RL methods that freeze the pretrained flow policy and run actor-critic over its input noise [51]. Yet, both have fundamental limits: residual methods correct only at the action level, ignoring the multi-step generative dynamics, while noise-space methods are bounded by the expressivity of the frozen flow policy.

Recently, Q-learning with Adjoint Matching (QAM) [25] addresses this by reformulating fine-tuning as a memoryless stochastic optimal control (SOC) problem: QAM uses a learned critic to control the sampling process toward higher-value actions via adjoint matching. While this resolves the multi-step sampling instability, critic-induced instability still remains. In off-policy RL, the learned critic is inevitably imperfect, thus the approximation errors compound through TD bootstrapping, where each value update depends on the critic’s noisy estimate at the next state, producing systematic

¹KAIST AI ²Seoul National University ³RLWRLD

Code: <https://github.com/yonghdong/trqam> Blog: <https://yonghdong.github.io/blog/trqam/>

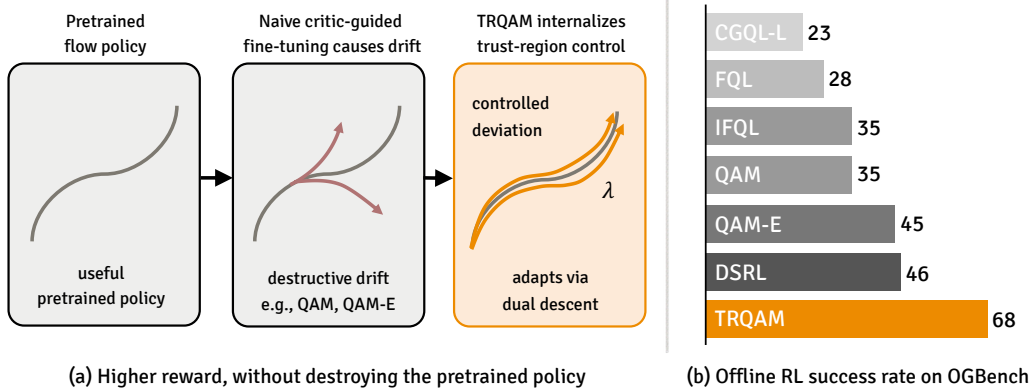


Figure 1: **TRQAM builds adaptive trust-region control into the SOC dynamics.** (a): Methods whose optimum admits an exponentially-tilted form (e.g., QAM, QAM-E [25]) suffer from destructive drift, where small critic errors can be exponentially amplified into large deviations from the pretrained prior (Lemma 1). TRQAM regulates this deviation through a trust-region parameter λ internalized in the SOC sampling dynamics. (b): Offline RL success rate across 50 OGBench [38] tasks. TRQAM outperforms adjoint-matching baselines (QAM, QAM-E) and other flow-policy fine-tuning paradigms (FQL [39], IFQL [22], DSRL [51], CGQL-L [8]) which lack such convergence guarantees.

overestimation [12]. Critic-guided policy updates can then amplify these errors into large deviations from the pretrained prior (see Lemma 1). QAM [25] acknowledges this and apply gradient clipping as a partial remedy, while calling for a *more principled method* beyond this heuristics. We empirically observe that gradient clipping does not prevent this fragility: the adjoint loss can become unstable and collapse task performance on Robomimic [32] (Figures 2, 10, and 11). Thus, the central challenge becomes how to improve downstream performance without destructive drift from the pretrained prior, similar to the trust-region principle in on-policy RL [46, 47].

We propose *Trust Region Q-Adjoint Matching* (TRQAM), an algorithm for stable off-policy fine-tuning of pretrained flow policies. TRQAM introduces a trust-region parameter λ directly into the stochastic optimal control (SOC) sampling dynamics and adapts it via projected dual descent to enforce a prescribed KL bound between the fine-tuned and pretrained policies. Our central theoretical result, proved using Girsanov’s theorem, shows that scaling the diffusion coefficient by $\sqrt{\lambda}$ makes the path-space KL between the controlled and pretrained sampling processes an explicit closed-form function of λ . As a result, the dual update enforces the target KL bound directly through the sampling dynamics, rather than softly imposing the constraint through a conventional loss-level KL regularizer.

This distinction matters in practice. Conventional loss-level KL regularization only competes with critic guidance at the loss level, allowing strong critic signals to push the realized KL far beyond the target bound and leave the policy vulnerable to collapse. In contrast, TRQAM tightly tracks the target bound throughout both offline and online training (Figures 5, 14, 15, and 16). The prescribed target KL bound therefore provides a practical control over how much the fine-tuned policy can deviate from the pretrained policy, and its best setting varies systematically with task structure (Section 4.3). Through experiments on 50 OGBench tasks, TRQAM consistently outperforms prior methods in both offline RL and offline-to-online RL. In particular, TRQAM achieves an overall offline RL success rate of 68%, substantially outperforming the strongest baseline at 46% (Table 1).

Contributions. We highlight the key contributions of our paper below:

- We identify the exponential amplification of critic errors as a fundamental fragility of fixed-temperature adjoint matching, formalized by Lemma 1 and confirmed empirically on Robomimic.
- We prove that scaling the diffusion coefficient by $\sqrt{\lambda}$ makes the path-space KL an exact function of λ via Girsanov (Theorem 1), turning λ into a principled trust-region parameter.
- We propose *Trust Region Q-Adjoint Matching* (TRQAM), which internalizes λ inside the SOC sampling dynamics and adapts it via projected dual descent to enforce a target KL bound at the sampling level rather than as a loss-level penalty.
- On 50 OGBench tasks, TRQAM consistently outperforms prior arts on offline RL and offline-to-online RL. Especially, TRQAM achieves an overall success rate of 68% in offline RL, demonstrating superior performance compared to 46% of strongest baseline.

2 Background

Reinforcement Learning. We consider a Markov Decision Process (MDP) [49] $(\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $r(s, a) : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is a reward function, $P(s' | s, a) : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ is a transition function, ρ_0 is the initial state distribution, and $\gamma \in [0, 1]$ is a discount factor. The objective is to learn a policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ that maximizes the expected discounted return $\mathbb{E}_{\rho_0, \pi, P}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$, where t indexes environment timesteps. To this end, we learn an action-value function $Q^\pi(s, a) = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k r(s_{t+k}, a_{t+k}) | s_t = s, a_t = a]$ used as a critic that guides policy improvement. We majorly focus on *off-policy fine-tuning* of a pretrained policy $\pi_{\text{base}}(\cdot | s)$, where transitions (s, a, r, s') used for updates are drawn from a replay buffer $\mathcal{D}$ collected by behavior policies different from the current π_θ .

Flow matching and flow policy. Flow Matching [2, 27] learns a velocity field $v_\theta(x, \tau)$ that transports samples from a simple source distribution $p_0 = \mathcal{N}(0, I_d)$ to a target distribution p_1 through an ODE:

$$dX_\tau = v_\theta(X_\tau, \tau) d\tau, \quad X_0 \sim p_0, \quad X_1 \sim p_1.$$

A standard choice is the optimal transport (OT) path, with straight-line interpolation $X_\tau = (1 - \tau)X_0 + \tau X_1$ between endpoint pairs $(X_0, X_1) \sim p_0 \otimes p_1$ and target velocity $\mathbb{E}[X_1 - X_0 | X_\tau = x]$. Then, the velocity field is trained by regressing v_θ against target velocity $X_1 - X_0$, giving

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\tau, X_0, X_1} \left[\|v_\theta(X_\tau, \tau) - (X_1 - X_0)\|^2 \right].$$

Note that a flow policy applies flow matching to model policy: given a state s , a flow policy $\pi_\theta(\cdot | s)$ samples actions by integrating $dX_\tau = v_\theta(X_\tau, \tau; s) d\tau$ from $X_0 \sim \mathcal{N}(0, I_d)$.

Stochastic optimal control and Q adjoint matching. Stochastic optimal control (SOC) is a framework that fine-tunes a pretrained flow policy by adding a drift perturbation u to its sampling dynamics, steering trajectories toward higher-reward regions without backpropagating through the multi-step sampling chain, a problem similar to backpropagation through time in RNNs [39, 59]. Domingo-Enrich et al. [10] showed that this SOC objective can be solved efficiently via a lean adjoint ODE, leading to the adjoint matching algorithm.

To formulate this, we first replace the deterministic flow ODE with an equivalent SDE whose distribution at every timestep $\tau \in [0, 1]$ coincides with that of the ODE [31]. While each trajectory might differ, the random variable X_τ at each timestep has the same distribution under both processes, including the terminal $X_1 \sim \pi_{\text{base}}$. Following Domingo-Enrich et al. [10], we adopt the memoryless OT schedule, and apply the SOC parameterization of Domingo-Enrich et al. [9] that explicitly separates a scalar $\sqrt{\lambda}$ from the diffusion coefficient, giving $\sqrt{\lambda} \sigma(\tau) = \sqrt{2(1 - \tau)/\tau}$. This gives the base and controlled SDEs

$$dX_\tau^{\text{base}} = b(X_\tau^{\text{base}}, \tau) d\tau + \sqrt{\lambda} \sigma(\tau) dB_\tau, \quad (1)$$

$$dX_\tau^u = (b(X_\tau^u, \tau) + \sigma(\tau) u(X_\tau^u, \tau)) d\tau + \sqrt{\lambda} \sigma(\tau) dB_\tau, \quad (2)$$

where the base SDE generates the pretrained policy and the control u steers the process away from it.

Given a terminal cost $g : \mathbb{R}^d \rightarrow \mathbb{R}$, SOC seeks the control u by optimizing

$$\min_u \mathbb{E}_{\mathbf{X} \sim \mathbb{P}^u} \left[\frac{1}{2} \int_0^1 \|u(X_\tau^u, \tau)\|^2 d\tau - g(X_1^u) \right] \quad \text{subject to Equation (2)}. \quad (3)$$

From a reinforcement learning perspective, taking X_1^u as the action at state s and critic $Q^\pi(s, \cdot)$ as the terminal cost g , the control u steers the pretrained policy toward critic-preferred actions, with the quadratic term penalizing deviation from π_{base} [10]. This instantiation is *Q Adjoint Matching* (QAM) [25], which solves (3) via adjoint matching [10]: the adjoint $\tilde{a}_\tau$ is computed by integrating

$$\tilde{a}_{\tau-h} = \tilde{a}_\tau + h \tilde{a}_\tau^\top \nabla_x \left(2v^{\text{base}}(X_\tau, \tau) - \frac{1}{\tau} X_\tau \right), \quad \tilde{a}_1 = -\nabla_{x_1} Q^\pi(s, X_1), \quad (4)$$

backwards in time, and the fine-tuned velocity field v_θ^{ft} , whose deviation from v^{base} parameterizes the control u , is updated by minimizing the adjoint-matching loss

$$\mathcal{L}_{\text{Adj-Match}}(\theta) = \sum_{\tau \in \{0, \dots, 1-h\}} \left\| \frac{2}{\sigma(\tau)} (v_\theta^{\text{ft}}(X_\tau, \tau) - v^{\text{base}}(X_\tau, \tau)) + \sigma(\tau) \tilde{a}_\tau \right\|^2, \quad (5)$$

without backpropagating through the sampling chain. However, QAM uses standard SOC dynamics without the $\sqrt{\lambda}$ -scaling, corresponding to the special case $\lambda = 1$ in our framework.

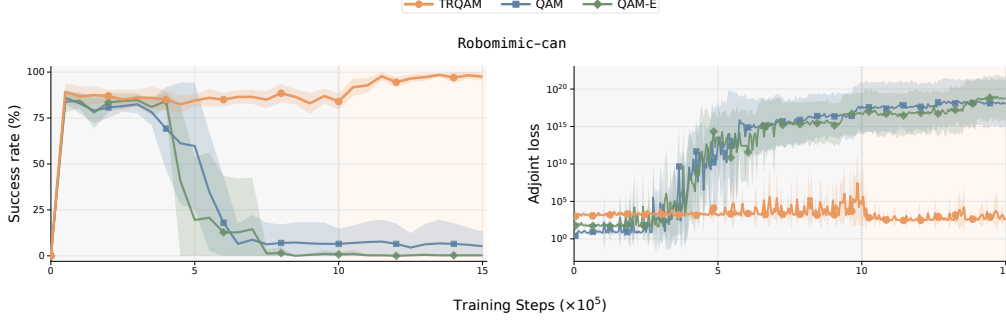


Figure 2: **Empirical fragility of fixed-temperature adjoint matching.** On Robomimic-can, the adjoint-matching loss in QAM and QAM-E grows above 10^{20} even with gradient clipping (min-max across seeds), driving task success from over 80% to near zero (± 1 standard deviation), while TRQAM remains stable. This collapse persists across most hyperparameter settings we tested on both Robomimic-lift and Robomimic-can; see Appendix Figures 10 and 11 for the full sweep.

3 Method

In this section, we introduce *Trust Region Q-Adjoint Matching* (TRQAM), a framework for stable off-policy fine-tuning of pretrained flow policies that adapts a trust-region parameter λ inside the stochastic optimal control sampling dynamics. By scaling the diffusion coefficient by $\sqrt{\lambda}$, TRQAM makes the path-space KL between the fine-tuned and pretrained processes an exact function of λ via Girsanov, turning a prescribed KL bound ε_{KL} into a structural constraint enforced at the sampling level. We develop our core contribution through investigating the following questions:

- Why fixed λ is fragile in off-policy RL? (Section 3.1)
- What does λ control when internalized in the SOC sampling dynamics? (Section 3.2)
- How do we adapt λ throughout training? (Section 3.3)
- Why must λ be internalized in the SOC sampling dynamics rather than added as a conventional KL regularization? (Section 3.4)

3.1 Why fixed λ is fragile in off-policy RL?

In off-policy RL, the learned critic is inevitably imperfect: approximation error compounds through bootstrapping and replay, and is especially severe under distributional shift [12, 23]. The central risk of critic-guided policy improvement is that *small critic errors can induce large policy deviations*. We formalize this in the following lemma, which applies to any updated policy that exponentially tilted from a base policy. Note that this includes QAM, since solving its memoryless SOC objective yields a terminal policy of this form [10, 25].

Lemma 1 (Exponential amplification of critic errors). *Fix a state $s \in \mathcal{S}$ and let $Q, \tilde{Q} : \mathcal{A} \rightarrow \mathbb{R}$ satisfy $\|Q - \tilde{Q}\|_\infty \leq \varepsilon$. Define the corresponding exponentially tilted distributions*

$$\pi_Q(a | s) \propto \pi_{\text{base}}(a | s) e^{\beta Q(a)}, \quad \pi_{\tilde{Q}}(a | s) \propto \pi_{\text{base}}(a | s) e^{\beta \tilde{Q}(a)},$$

where $\beta > 0$ is the inverse temperature. Then the following inequalities hold:

$$D_{\text{KL}}(\pi_Q \| \pi_{\tilde{Q}}) \leq 2\beta\varepsilon, \quad \text{TV}(\pi_Q, \pi_{\tilde{Q}}) \leq \frac{1}{2}(e^{2\beta\varepsilon} - 1).$$

Proof. See Appendix D.1 for the full proof. □

The total-variation bound is exponential in $\beta\varepsilon$: large β amplifies critic errors into large policy deviations, while small β suppresses both critic errors and useful improvement. The lemma thus formalizes a fundamental tension in critic-guided updates: no single fixed β can simultaneously exploit a reliable critic and protect against an unreliable one. We observe this empirically on Robomimic [32]: fixed-temperature QAM and QAM-E exhibit diverging adjoint loss and collapsing

task success even with gradient clipping (Figure 2). An analogous tension arises in our SOC setting, where λ governs the size of the trust region: small λ permits aggressive deviation from π_{base} (corresponding to large β), while large λ keeps the controlled sampler close to it. Before we can adapt λ , we need a quantitative link between λ and the deviation from π_{base} , which we establish next.

3.2 λ as a trust-region parameter

To adapt λ in a principled way, we first establish what λ controls. Note that the diffusion coefficient is scaled by $\sqrt{\lambda}$ in Equation (2), following the parameterization of SOCM [9]. Its consequence under change-of-measure, however, has not been made explicit. We now derive this consequence: under this scaling, Girsanov’s theorem yields an exact identity between the quadratic control cost and the path-space KL between the fine-tuned and base trajectory distributions, in which λ appears explicitly as the inverse coefficient.

Theorem 1 (SOC control cost = path-space KL). *Let $\mathbb{P}^u$ and $\mathbb{P}^{\text{base}}$ denote the distributions over trajectories induced by the controlled dynamics (2) and the base dynamics (1). Then,*

$$D_{\text{KL}}(\mathbb{P}^u \parallel \mathbb{P}^{\text{base}}) = \mathbb{E}_{\mathbf{x} \sim \mathbb{P}^u} \left[\frac{1}{2\lambda} \int_0^1 \|u(X_\tau, \tau)\|^2 d\tau \right]. \quad (6)$$

Proof. See Appendix D.2 for the full proof. $\square$

While Theorem 1 ties λ to the path-space KL, the quantity we ultimately care about is the deviation of the terminal action distribution $\pi_\theta(\cdot \mid s)$ from $\pi_{\text{base}}(\cdot \mid s)$. The following proposition shows that controlling the path-space KL suffices: it provides an upper bound on the terminal KL.

Proposition 1 (Terminal KL upper-bounded by path-space KL). *Let $\mathbb{P}^u$ and $\mathbb{P}^{\text{base}}$ denote the distributions over trajectories induced by the controlled dynamics (2) and the base dynamics (1), and let $\pi_\theta(\cdot \mid s)$ and $\pi_{\text{base}}(\cdot \mid s)$ denote the corresponding terminal action distributions at $\tau = 1$. Then,*

$$D_{\text{KL}}(\pi_\theta(\cdot \mid s) \parallel \pi_{\text{base}}(\cdot \mid s)) \leq D_{\text{KL}}(\mathbb{P}^u \parallel \mathbb{P}^{\text{base}}). \quad (7)$$

Proof. See Appendix D.3 for the full proof. $\square$

Informally, the three results form a chain that links the trust-region parameter λ to the amplification of critic errors. Theorem 1 ties λ to the path-space KL between the controlled and base trajectories. Proposition 1 shows that this path-space KL upper-bounds the terminal-policy KL $D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{base}})$. Lemma 1 bounds this terminal-policy KL by $2\beta\varepsilon$, where β is the inverse temperature in the exponential tilting of π_{base} by the critic and ε is the critic approximation error.

Remark: Connection between trust-region parameter λ and inverse temperature β

$$\frac{1}{\lambda} \underset{\text{Thm. 1}}{\propto} \underbrace{D_{\text{KL}}(\mathbb{P}^u \parallel \mathbb{P}^{\text{base}})}_{\text{path-space KL}} \underset{\text{Prop. 1}}{\geq} \underbrace{D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{base}})}_{\text{terminal KL}} \underset{\text{Lem. 1}}{\lesssim} \underbrace{\beta\varepsilon}_{\text{critic-error bound}}.$$

Increasing λ shrinks terminal KL, effectively reducing the critic guidance strength β and tightening the bound on critic-error amplification. A single scalar λ therefore adaptively balances exploiting the critic and staying close to π_{base} .

3.3 Adaptive λ via projected dual descent

To keep the realized path-space KL within a target bound ε_{KL} , we need a tractable KL estimator and a principled rule for updating λ based on it. We use the fact that the discretized memoryless OT sampler with step size h and diffusion schedule $g(\tau) = \sqrt{2(1-\tau)/\tau}$ is a Markov chain whose Gaussian transitions share the same covariance, so per-step KL divergences admit a closed form.

Algorithm 1 QAM

Require: v^{base} , training step N
1: Init $v_\theta^{\text{ft}} \leftarrow v^{\text{base}}, \lambda = 1$
2: **for** $n = 0, \dots, N - 1$ **do**
3: Sample X via Eq. (2) with $v_\theta^{\text{ft}}, \lambda$
4: Solve adjoint ODE (4)
5: $\theta \leftarrow \theta - \nabla_\theta \mathcal{L}_{\text{Adj-Match}}$ via Eq. (5) with σ
6:
7:
8: **end for**

Algorithm 2 TRQAM (ours)

Require: v^{base} , training step N , KL budget ε_{KL}
1: Init $v_\theta^{\text{ft}} \leftarrow v^{\text{base}}, \lambda_0, \bar{D}_0$, dual stepsize η_λ
2: **for** $n = 0, \dots, N - 1$ **do**
3: Sample X via Eq. (2) with $v_\theta^{\text{ft}}, \lambda_n$
4: Solve adjoint ODE (4)
5: $\theta \leftarrow \theta - \nabla_\theta \mathcal{L}_{\text{Adj-Match}}$ via Eq. (5) with σ_n
6: Estimate $\hat{D}_n$ via Eq. (8); EMA $\bar{D}_n$
7: $\lambda_{n+1} \leftarrow \max\{0, \lambda_n + \eta_\lambda (\bar{D}_n - \varepsilon_{\text{KL}})\}$
8: **end for**

Summing these per-step KLs along a trajectory approximately recovers the path-space KL, which we estimate via Monte Carlo over sampled trajectories (derivation in Appendix E):

$$\hat{D}_n = \mathbb{E}_{\mathbf{X} \sim \mathbb{P}^u} \left[\sum_{k=0}^{K-1} \frac{2h}{g(\tau_k)^2} \|v_\theta^{\text{ft}}(X_{\tau_k}, \tau_k) - v^{\text{base}}(X_{\tau_k}, \tau_k)\|^2 \right], \quad (8)$$

To reduce variance, we smooth $\hat{D}_n$ with an exponential moving average: $\bar{D}_n \leftarrow (1 - \rho) \bar{D}_{n-1} + \rho \hat{D}_n$. Given this estimator, we adapt λ by interpreting it as the dual variable of a KL-constrained improvement problem $\max_u \mathbb{E}[Q^\pi(X_1^u)]$ subject to $D_{\text{KL}}(\mathbb{P}^u \| \mathbb{P}^{\text{base}}) \leq \varepsilon_{\text{KL}}$, and apply projected dual descent with a fixed step size $\eta_\lambda > 0$ (derivation in Appendix F):

$$\lambda_{n+1} \leftarrow \max\{0, \lambda_n + \eta_\lambda (\bar{D}_n - \varepsilon_{\text{KL}})\}. \quad (9)$$

When the realized KL exceeds the bound, λ rises and the controlled dynamics become more conservative; when it falls below, λ decreases and allows more aggressive improvement. Algorithm 2 summarizes the resulting TRQAM update, where the adaptive trust-region components are highlighted in blue. Full algorithm can be found in Appendix A.

3.4 Internal vs. external KL regularization

The dual update in Equation (9) can be combined with the SOC objective in two different approaches:

$$\underbrace{\min_\theta \mathcal{L}_{\text{Adj-Match}}(\theta) + \lambda \cdot \bar{D}_n(\theta)}_{\text{External: } \lambda \text{ is a regularization weight}} \quad \text{vs.} \quad \underbrace{\min_\theta \mathcal{L}_{\text{Adj-Match}}(\theta) \text{ s.t. SDE uses } \sqrt{\lambda} \sigma(\tau)}_{\text{Internal (TRQAM): } \lambda \text{ appears in the SOC sampling dynamics}}. \quad (10)$$

For the internal form (right), $\sqrt{\lambda} \sigma(\tau) = \sqrt{2(1-\tau)/\tau}$ is fixed by the OT schedule, thus adjusting λ changes $\sigma(\tau)$ and reshapes the entire controlled SDE, including its drift term $b(x, \tau) + \sigma(\tau) u(x, \tau)$ in Equation (2). Intuitively, increasing λ shrinks $\sigma(\tau)$, which weakens the control contribution $\sigma(\tau) u(x, \tau)$ to the drift and pulls the controlled SDE toward the base dynamics. By Theorem 1, the realized path-space KL is an exact function of λ , and the dual update directly enforces the trust region through the sampling dynamics. For the external form (left), λ enters only as a coefficient on a loss penalty, and has limitations in enforcing the target. As such, under strong critic guidance, the realized KL can drift far from this target. We refer to Table 6 in Appendix G, where we summarize these structural differences side-by-side. Also, we validate this distinction empirically in Section 4.2.

4 Experiments

We evaluate TRQAM on off-policy fine-tuning of pretrained flow policies in the offline-to-online setting. We use OGBench [38] (50 tasks) for main comparison, and Robomimic [32] for ablation and mechanism studies. All methods share the same pretrained flow policy and training schedule.

Setup. OGBench [38] is an offline goal-conditioned RL benchmark spanning 10 suites, from which we evaluate on 50 tasks. While OGBench is originally designed for offline goal-conditioned RL, we use its reward-based single-task variants. Robomimic [32] is a demonstration based manipulation benchmark used to test the stability. For all manipulation tasks (e.g., OGBench’s scene, cube, puzzle suites and all Robomimic tasks), we use action-chunked policies with chunk size $h = 5$ [26].

Table 1: **Offline RL on 50 OGBench [38] tasks at 1M training steps (8 seeds).** Mean success rate (%) with ± 1 standard deviation. (per-task breakdown across all 50 tasks in Table 7).

		al	ag	hm	hl	scene	p33	p44	c2	c3	c4	all
		5 tasks	5 tasks	5 tasks	5 tasks	5 tasks	5 tasks	5 tasks	5 tasks	5 tasks	5 tasks	50 tasks
BACKPROP	FQL	38 $\pm$ 9	2 $\pm$ 6	74 $\pm$ 5	2 $\pm$ 1	70 $\pm$ 5	25 $\pm$ 10	9 $\pm$ 7	44 $\pm$ 4	7 $\pm$ 5	9 $\pm$ 5	28
GUIDANCE	CGQL-L	48 $\pm$ 7	7 $\pm$ 5	57 $\pm$ 2	6 $\pm$ 3	58 $\pm$ 1	0 $\pm$ 0	0 $\pm$ 0	55 $\pm$ 2	0 $\pm$ 1	1 $\pm$ 1	23
POST PROCESSING	DSRL	53 $\pm$ 2	1 $\pm$ 1	53 $\pm$ 10	1 $\pm$ 1	80 $\pm$ 0	100 $\pm$ 0	61 $\pm$ 8	72 $\pm$ 4	34 $\pm$ 6	9 $\pm$ 3	46
	IFQL	29 $\pm$ 8	12 $\pm$ 3	93 $\pm$ 2	30 $\pm$ 7	36 $\pm$ 1	64 $\pm$ 4	42 $\pm$ 4	9 $\pm$ 2	24 $\pm$ 7	6 $\pm$ 3	35
ADJOINT MATCHING	QAM	62 $\pm$ 9	29 $\pm$ 4	64 $\pm$ 7	4 $\pm$ 3	64 $\pm$ 4	15 $\pm$ 3	1 $\pm$ 1	71 $\pm$ 2	19 $\pm$ 6	18 $\pm$ 3	35
	QAM-E	86 $\pm$ 3	6 $\pm$ 8	60 $\pm$ 6	4 $\pm$ 5	63 $\pm$ 6	89 $\pm$ 4	54 $\pm$ 8	71 $\pm$ 3	11 $\pm$ 4	9 $\pm$ 3	45
OURS	TRQAM	89 $\pm$ 4	41 $\pm$ 4	84 $\pm$ 3	36 $\pm$ 4	79 $\pm$ 1	100 $\pm$ 0	99 $\pm$ 1	81 $\pm$ 3	50 $\pm$ 5	19 $\pm$ 5	68

On OGBench, we compare our method against six off-policy fine-tuning baselines: **FQL** [39], **CGQL-L** [8], **DSRL** [51], **IFQL** [14], and **QAM / QAM-E** [25]. On Robomimic, we focus on the adjoint-matching variants (*i.e.*, QAM and QAM-E), and DSRL as a non-adjoint reference. We refer to Appendix B for details on each baseline. Since trust-region fine-tuning regulates deviation from a pretrained prior, this prior must itself encode meaningful behavior. Therefore, all methods are pretrained for 300K steps with behavior cloning, and run offline-to-online fine-tuning for each 1M step. We report average success rate (%) over 8 seeds, and the detailed hyperparameters are in Appendix C.3.

Abbreviations. We evaluate on 10 OGBench task suites and abbreviate their names in tables and figures for compactness: puzzle-4x4 (p44), cube-double (c2), cube-triple (c3), cube-quadruple (c4), scene, humanoidmaze-medium (hm), humanoidmaze-large (hl), antmaze-large (al), antmaze-giant (ag), and puzzle-3x3 (p33).

4.1 Main results on offline and offline-to-online RL

Table 1 reports offline success rates after 1M training steps. TRQAM achieves 68% aggregate success across 50 tasks, improving on QAM (35%) by 33 points, on its strongest variant QAM-E (45%) by 23 points, and on the strongest non-adjoint baseline DSRL (46%) by 22 points, with the largest gains on long-horizon and combinatorial suites. This lead is sustained through the offline-to-online transition: per-task curves in Appendix Figures 17 and 18 show that TRQAM remains the strongest method through 500K steps of online fine-tuning.

A natural explanation for TRQAM’s gains is that it leverages the pretrained policy more effectively. Figure 3 tests this by running TRQAM, QAM-E, and QAM both from a pretrained flow policy (dashed) and from scratch (solid) under an identical offline-to-online protocol. The contrast is clear: TRQAM benefits substantially from the pretrained prior, reaching high success much earlier than its scratch counterpart, while QAM and QAM-E show little to no benefit from the same pretrained initialization, with their pretrained and scratch curves remaining close throughout training.

4.2 What mechanism drives its gains?

To isolate what drives the asymmetry above, we compare three variants differing only in how λ is regulated: **QAM** (constant λ in our framework), **QAM + External KL** (adaptive λ as a conventional KL regularization loss), and **TRQAM** (adaptive λ internalized in the SOC sampling dynamics). This isolates two design axes: *adaptation* (constant KL vs. adaptive KL) and *internalization* (external loss penalty vs. internalized SOC sampling dynamics).

1. Adaptative KL outperforms constant KL. Both adaptive KL variants, TRQAM and QAM with external KL regularization, substantially outperform QAM on cube-triple-task1 and humanoidmaze-medium-task1 (Figure 4). The same asymmetry appears on Robomimic, where QAM exhibits the diverging adjoint loss and collapsing task success of Figure 2 while both adaptive variants remain stable (Appendix H.4). These results are consistent with Lemma 1’s exponential amplification of critic errors under fixed temperature.

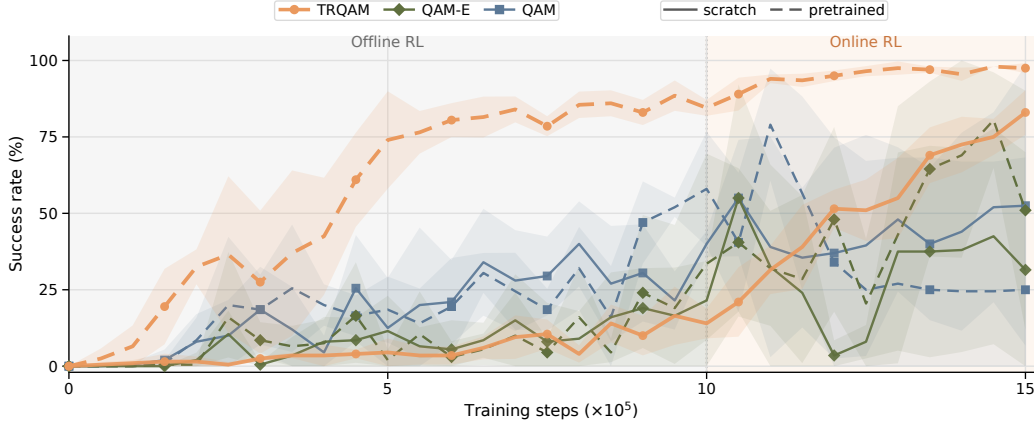


Figure 3: **Pretraining alone is not sufficient.** On humanoidmaze-medium-task1, each algorithm is run from a pretrained flow policy (dashed) and from scratch (solid); shaded regions denote standard deviation across seeds. TRQAM benefits substantially from the pretrained prior, while QAM and QAM-E show little to no benefit from the same pretrained initialization, with their pretrained and scratch curves remaining close throughout training.

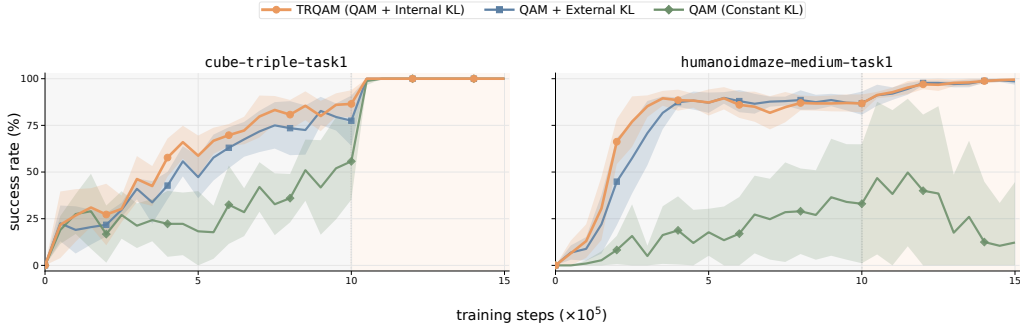


Figure 4: **Adaptation is necessary.** Both adaptive variants (TRQAM and QAM + External KL) outperform QAM (constant λ) on cube-triple-task1 and humanoidmaze-medium-task1, consistent with Lemma 1: a fixed temperature can amplify critic errors into policy deviations, while adaptation can mitigate this. Shaded regions denote standard deviation across seeds.

2. Internal KL outperforms external KL regularization. Among the two adaptive variants, only TRQAM enforces the prescribed KL budget. On Robomimic-lift and Robomimic-can with $\varepsilon_{KL} = 0.1$, TRQAM tightly tracks the bound throughout training, while QAM + External KL lets the realized KL drift above ε_{KL} with corresponding success-rate degradation (Figure 5). By Theorem 1, only the internal parameterization ties λ to the realized KL through an exact identity, whereas an external penalty enters only as an additive loss term that strong critic guidance can override. The same pattern holds across all three Robomimic tasks and six KL budgets (Appendix Figures 14, 15, and 16). These violations are consistent with Lemma 1’s exponential amplification of critic errors.

4.3 Sensitivity Analysis

Remark that ε_{KL} is the most important hyperparameter, and varying it produces predictable changes in success rate. We sweep ε_{KL} from 0.5 to 4 in steps of 0.5 on four representative OGBench tasks (humanoidmaze-medium-task1, humanoidmaze-large-task1, cube-double-task2, cube-triple-task2), with the same training setup as Section 4.1. Figure 6 shows two patterns. First, success rate changes smoothly with ε_{KL} on every task. Second, tight budgets are best across all four tasks. The similar pattern holds across most of the ten OGBench domains in Appendix H.3, with puzzle-4x4 as the exception where larger budgets monotonically improve performance, in keeping with its notably larger state space. Because TRQAM tightly enforces the chosen budget (Section 4.2), tuning ε_{KL} to task structure produces predictable, controlled behavior, so adapting TRQAM to a new environment comes down to choosing ε_{KL} based on task structure.

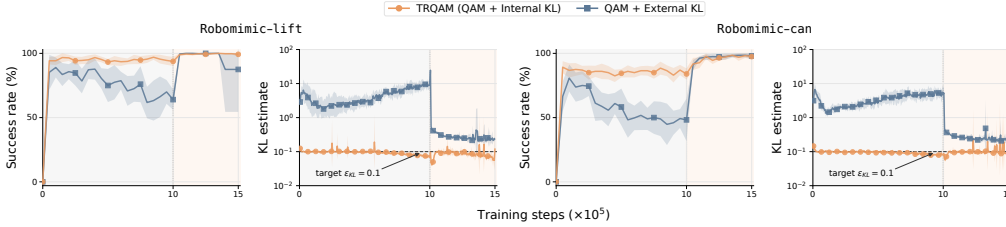


Figure 5: **Internalization is necessary.** On Robomimic-lift & Robomimic-can with $\epsilon_{KL} = 0.1$, TRQAM tightly tracks the target KL bound throughout training, while QAM + External KL lets the realized KL drift well above it, with corresponding success rate degradation. By Theorem 1, only the internal parameterization (TRQAM) ties λ to the realized KL through an exact identity; the external loss penalty (QAM + External KL) can be overridden by strong critic guidance. Shaded regions denote standard deviation across seeds.

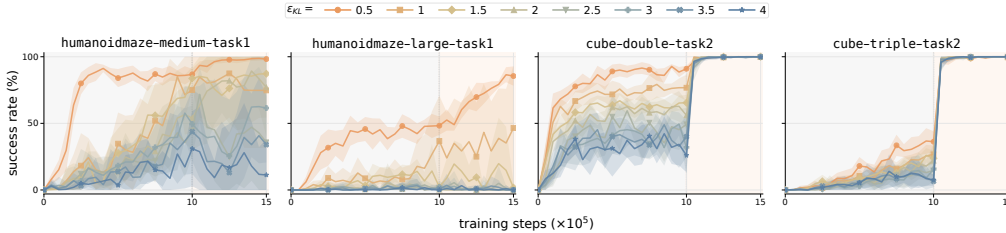


Figure 6: **Sensitivity Analysis.** Success-rate curves on four OGBench tasks under varying KL budgets. Two patterns emerge. First, success rate changes smoothly with ϵ_{KL} on every task, making the budget a predictable knob. Second, tight budgets are best across all four tasks, with the optimum tracking task structure.

5 Related Work

Offline-to-online RL. Pretraining a policy and value function on offline data and then fine-tuning online is a standard recipe for sample-efficient RL [3, 15, 16, 22–24, 30, 34, 35, 43, 50]. A central challenge is the distribution shift at the transition, which destabilizes the value function and induces catastrophic forgetting [3, 35, 53], motivating trust-region-style constraints during fine-tuning.

Fine-tuning flow and diffusion policies. Flow matching and diffusion policies [7, 27] parameterize multi-modal action distributions and are increasingly pretrained at scale [4, 5, 18, 19]. A growing body of work trains such policies with RL [6, 11, 14, 28, 39, 44, 51, 52, 55, 56, 58, 59], each navigating a tradeoff between policy expressivity, computational cost, and training stability.

KL trust regions in RL. KL regularization stabilizes policy updates as a hard constraint or soft penalty [1, 41, 46, 47, 54], sometimes with strength adapted via dual updates (e.g., SAC [13] for entropy). These approaches enforce the constraint via auxiliary losses external to the policy.

6 Conclusion

We introduce *Trust Region Q-Adjoint Matching* (TRQAM), a stable off-policy fine-tuning method for pretrained flow policies. TRQAM adapts a trust-region parameter λ inside the SOC sampling dynamics: scaling the diffusion by $\sqrt{\lambda}$ makes the path-space KL an exact function of λ via Girsanov (Theorem 1), so dual descent on λ enforces the target bound at the sampling level rather than through a loss-level penalty. Across 50 OGBench tasks, TRQAM improves the strongest baseline by 22 points, with the largest gains on long-horizon and combinatorial domains; on Robomimic, it remains stable where fixed-temperature adjoint matching collapses. In the spirit of TRPO and PPO for on-policy RL, we hope TRQAM offers an analogous trust-region stabilization for off-policy fine-tuning of pretrained flow policies.

Limitations. Computing the adjoint matching loss requires a vector-Jacobian product (VJP) through the velocity field at each step of the backward ODE; this VJP cost scales with model size.

Acknowledgments and Disclosure of Funding

This work was supported by Institute for Information & communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (RS-2019-II190075, Artificial Intelligence Graduate School Program(KAIST)). We thank RLWRLD Inc. for providing compute resources to conduct experiments in this work.

References

- [1] A. Abdolmaleki, J. T. Springenberg, Y. Tassa, R. Munos, N. Heess, and M. Riedmiller. Maximum a posteriori policy optimisation. In *International Conference on Learning Representations*, 2018.
- [2] M. S. Albergo, N. M. Boffi, and E. Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. In *Journal of Machine Learning Research*, 2025.
- [3] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning*, 2023.
- [4] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots, 2025. URL <https://arxiv.org/abs/2503.14734>.
- [5] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0 : A vision-language-action flow model for general robot control. In *Robotics: Science and Systems*, 2025.
- [6] K. Chen, Z. Liu, T. Zhang, Z. Guo, S. Xu, H. Lin, H. Zang, X. Li, Q. Zhang, Z. Yu, G. Fan, T. Huang, Y. Wang, and C. Yu. π_{RL} : Online rl fine-tuning for flow-based vision-language-action models, 2026. URL <https://arxiv.org/abs/2510.25889>.
- [7] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.
- [8] P. Dhariwal and A. Nichol. Diffusion models beat gans on image synthesis. In *Advances in Neural Information Processing Systems*, 2021.
- [9] C. Domingo-Enrich, J. Han, B. Amos, J. Bruna, and R. T. Q. Chen. Stochastic optimal control matching. In *Advances in Neural Information Processing Systems*, 2024.
- [10] C. Domingo-Enrich, M. Drozdal, B. Karrer, and R. T. Q. Chen. Adjoint matching: Fine-tuning flow and diffusion generative models with memoryless stochastic optimal control. In *International Conference on Learning Representations*, 2025.
- [11] P. Dong, Q. Li, D. Sadigh, and C. Finn. Expo: Stable reinforcement learning with expressive policies. In *International Conference on Learning Representations*, 2026.
- [12] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, 2018.
- [13] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, 2018.
- [14] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine. Idql: Implicit q-learning as an actor-critic method with diffusion policies, 2023. URL <https://arxiv.org/abs/2304.10573>.
- [15] T. Hester, M. Vecerik, O. Pietquin, M. Lanctot, T. Schaul, B. Piot, D. Horgan, J. Quan, A. Sendonaris, G. Dulac-Arnold, I. Osband, J. Agapiou, J. Z. Leibo, and A. Gruslys. Deep q-learning from demonstrations. In *AAAI Conference on Artificial Intelligence*, 2018.

- [16] H. Hu, S. Mirchandani, and D. Sadigh. Imitation bootstrapped reinforcement learning. In *International Conference on Learning Representations*, 2024.
- [17] C.-Y. Hung, N. Majumder, H. Deng, L. Renhang, Y. Ang, A. Zadeh, C. Li, D. Herremans, Z. Wang, and S. Poria. Nora-1.5: A vision-language-action model trained using world model- and action-based preference rewards, 2025. URL <https://arxiv.org/abs/2511.14659>.
- [18] P. Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, D. Driess, M. Equi, A. Esmail, Y. Fang, C. Finn, C. Glossop, T. Godden, I. Goryachev, L. Groom, H. Hancock, K. Hausman, G. Hussein, B. Ichter, S. Jakubczak, R. Jen, T. Jones, B. Katz, L. Ke, C. Kuchi, M. Lamb, D. LeBlanc, S. Levine, A. Li-Bell, Y. Lu, V. Mano, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, C. Sharma, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, W. Stoeckle, A. Swerdlow, J. Tanner, M. Torne, Q. Vuong, A. Walling, H. Wang, B. Williams, S. Yoo, L. Yu, U. Zhilinsky, and Z. Zhou. $\pi_{0.6}^*$: a vla that learns from experience, 2025. URL <https://arxiv.org/abs/2511.14759>.
- [19] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Conference on Robot Learning*, 2025.
- [20] T. Jiang, T. Yuan, Y. Liu, C. Lu, J. Cui, X. Liu, S. Cheng, J. Gao, H. Xu, and H. Zhao. Galaxea open-world dataset and g0 dual-system vla model, 2025. URL <https://arxiv.org/abs/2509.00576>.
- [21] C. Kim, H. Lee, Y. Seo, K. Lee, and Y. Zhu. Deas: Detached value learning with action sequence for scalable offline rl. In *International Conference on Learning Representations*, 2026.
- [22] I. Kostrikov, A. Nair, and S. Levine. Offline reinforcement learning with implicit q-learning. In *International Conference on Learning Representations*, 2022.
- [23] A. Kumar, A. Zhou, G. Tucker, and S. Levine. Conservative q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, 2020.
- [24] K. Lei, Z. He, C. Lu, K. Hu, Y. Gao, and H. Xu. Uni-o4: Unifying online and offline deep reinforcement learning with multi-step on-policy optimization. In *International Conference on Learning Representations*, 2024.
- [25] Q. Li and S. Levine. Q-learning with adjoint matching. In *International Conference on Learning Representations*, 2026.
- [26] Q. Li, Z. Zhou, and S. Levine. Reinforcement learning with action chunking. In *Advances in Neural Information Processing Systems*, 2025.
- [27] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.
- [28] J. Liu, G. Liu, J. Liang, Y. Li, J. Liu, X. Wang, P. Wan, D. Zhang, and W. Ouyang. Flow-grpo: Training flow matching models via online rl. In *Advances in Neural Information Processing Systems*, 2025.
- [29] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*, 2023.
- [30] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine. Serl: A software suite for sample-efficient robotic reinforcement learning. In *IEEE International Conference on Robotics and Automation*, 2024.
- [31] N. Ma, M. Goldstein, M. S. Albergo, N. M. Boffi, E. Vanden-Eijnden, and S. Xie. Sit: Exploring flow and diffusion-based generative models with scalable interpolant transformers. In *European Conference on Computer Vision*, 2024.

- [32] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *Conference on Robot Learning*, 2021.
- [33] V. Myers, B. Zheng, B. Eysenbach, and S. Levine. Offline goal-conditioned reinforcement learning with quasimetric representations. In *Advances in Neural Information Processing Systems*, 2025.
- [34] A. Nair, A. Gupta, M. Dalal, and S. Levine. Awac: Accelerating online reinforcement learning with offline datasets. In *International Conference on Learning Representations*, 2021.
- [35] M. Nakamoto, Y. Zhai, A. Singh, M. S. Mark, Y. Ma, C. Finn, A. Kumar, and S. Levine. Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning. In *Advances in Neural Information Processing Systems*, 2023.
- [36] N. Nüsken and L. Richter. Solving high-dimensional Hamilton–Jacobi–Bellman pdes using neural networks: perspectives from the theory of controlled diffusions and measures on path space. *Partial differential equations and applications*, 2:1–48, 2021.
- [37] B. Øksendal. *Stochastic differential equations*. Springer, 2003.
- [38] S. Park, K. Frans, B. Eysenbach, and S. Levine. Ogbench: Benchmarking offline goal-conditioned rl. In *International Conference on Learning Representations*, 2025.
- [39] S. Park, Q. Li, and S. Levine. Flow q-learning. In *International Conference on Machine Learning*, 2025.
- [40] A. Parsian and S. Kirmani. Estimation under linex loss function. In *Handbook of applied econometrics and statistical inference*, pages 75–98. CRC Press, 2002.
- [41] X. B. Peng, A. Kumar, G. Zhang, and S. Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. In *International Conference on Learning Representations*, 2021.
- [42] Y. Polyanskiy and Y. Wu. *Information Theory: From Coding to Learning*. Cambridge University Press, 2025.
- [43] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. In *Robotics: Science and Systems*, 2018.
- [44] A. Z. Ren, J. Lidard, L. L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz. Diffusion policy policy optimization. In *International Conference on Learning Representations*, 2025.
- [45] M. Reuss, H. Zhou, M. Rühle, Ö. E. Yağmurlu, F. Otto, and R. Lioutikov. FLOWER: Democratizing generalist robot policies with efficient vision-language-flow models. In *Conference on Robot Learning*, 2025.
- [46] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel. Trust region policy optimization. In *International Conference on Machine Learning*, 2015.
- [47] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- [48] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene. Smolvla: A vision-language-action model for affordable and efficient robotics, 2025. URL <https://arxiv.org/abs/2506.01844>.
- [49] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [50] M. Vecerik, T. Hester, J. Scholz, F. Wang, O. Pietquin, B. Piot, N. Heess, T. Rothörl, T. Lampe, and M. Riedmiller. Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards, 2018. URL <https://arxiv.org/abs/1707.08817>.

- [51] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine. Steering your diffusion policy with latent space reinforcement learning. In *Conference on Robot Learning*, 2025.
- [52] Z. Wang, J. J. Hunt, and M. Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. In *International Conference on Learning Representations*, 2023.
- [53] M. Wołczyk, B. Cupiał, M. Ostaszewski, M. Bortkiewicz, M. Zając, R. Pascanu, Ł. Kuciński, and P. Miłoś. Fine-tuning reinforcement learning models is secretly a forgetting mitigation problem. In *International Conference on Machine Learning*, 2024.
- [54] Y. Wu, G. Tucker, and O. Nachum. Behavior regularized offline reinforcement learning. In *International Conference on Learning Representations*, 2020.
- [55] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. J. Fan, G. Shi, and Y. Zhu. Self-improving vision-language-action models with data generation via residual rl. In *International Conference on Learning Representations*, 2026.
- [56] C. Xu, J. T. Springenberg, M. Equi, A. Amin, A. Esmail, S. Levine, and L. Ke. Rl token: Bootstrapping online rl with vision-language-action models, 2026. URL <https://arxiv.org/abs/2604.23073>.
- [57] A. Zhai, B. Liu, B. Fang, C. Cai, E. Ma, E. Yin, H. Wang, H. Zhou, J. Wang, L. Shi, L. Liang, M. Wang, Q. Wang, R. Gan, R. Yu, S. Li, S. Liu, S. Chen, V. Chen, and Z. Xu. Igniting vlms toward the embodied space, 2025. URL <https://arxiv.org/abs/2509.11766>.
- [58] T. Zhang, C. Yu, S. Su, and Y. Wang. Reinflow: Fine-tuning flow matching policy with online reinforcement learning. In *Advances in Neural Information Processing Systems*, 2025.
- [59] Y. Zhang, S. Yu, T. Zhang, M. Guang, H. Hui, K. Long, Y. Wang, C. Yu, and W. Ding. Sac flow: Sample-efficient reinforcement learning of flow-based policies via velocity-reparameterized sequential modeling. In *International Conference on Learning Representations*, 2026.
- [60] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng, T. Wang, Y.-Q. Zhang, J. Liu, and X. Zhan. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *International Conference on Learning Representations*, 2026.

Contents

A	Algorithm	15
B	Baselines	15
C	Experimental details	17
C.1	Domains and tasks	17
C.2	Compute resources	17
C.3	Hyperparameters	18
D	Proofs	20
D.1	Proof of Lemma 1: exponential amplification of critic errors	20
D.2	Proof of Theorem 1: path-space KL identity	21
D.3	Proof of Proposition 1: Terminal KL upper-bounded by path-space KL	22
E	Path-space KL surrogate under OT memoryless discretization	22
F	KL-budgeted improvement: primal–dual derivation	23
G	Internal vs. external KL regularization: detailed comparison	24
H	Additional experiments	25
H.1	TRQAM tightly enforces the prescribed KL budget	25
H.2	Time-varying KL budget	26
H.3	Sensitivity Analysis	26
H.4	Stability stress test on Robomimic	26
I	Broader impacts	27

A Algorithm

Algorithm 3 Trust Region Q-Adjoint Matching (TRQAM) for fine-tuning Flow Matching policies. **Blue** marks TRQAM additions over QAM [25].

Input: replay buffer $\mathcal{D}$; v^{base} : pretrained (behavior) velocity field; v_θ^{ft} : fine-tuned velocity field; Q_ϕ : critic function; step size h ; **KL budget** ε_{KL} ; **dual stepsize** η_λ ; **EMA coefficient** ρ ; fine-tuning iterations N .

Initialize: $v_\theta^{\text{ft}} \leftarrow v^{\text{base}}$ with parameters θ ; $\lambda_0 > 0$; $\overline{D}_0 \leftarrow 0$.

Memoryless SDE: $\sqrt{\lambda_n} \sigma_n(\tau) = g(\tau)$ for all n , where $g(\tau) := \sqrt{2(1-\tau)/\tau}$

for $n \in \{0, \dots, N-1\}$ **do**

 Sample a batch $\mathcal{B} = \{(s_i, a_i, r_i, s'_i)\}$ from $\mathcal{D}$

Critic update: Optimize ϕ w.r.t.

$$\mathcal{L}(\phi) = \frac{1}{|\mathcal{B}|} \sum_{(s,a,r,s') \in \mathcal{B}} [Q_\phi(s, a) - r - \gamma Q_\phi(s', a' \sim \pi_\theta(\cdot | s'))]^2 \quad \triangleright \text{TD backup} \quad (11)$$

Policy update: For each state $s \in \mathcal{B}$, sample a trajectory $\mathbf{X} = (X_\tau)_{\tau \in \{0, h, \dots, 1\}}$ via the memoryless Euler scheme:

$$X_{\tau+h} = X_\tau + h \left(2v_\theta^{\text{ft}}(s, X_\tau, \tau) - \frac{1}{\tau} X_\tau \right) + \sqrt{h} \underbrace{\sqrt{\lambda_n} \sigma_n(\tau)}_{=g(\tau)} \varepsilon_\tau, \quad \varepsilon_\tau, X_0 \sim \mathcal{N}(0, I). \quad (12)$$

 Compute the critic's action gradient: $\tilde{a}_1 \leftarrow -\nabla_{X_1} Q_\phi(s, X_1)$.

 Solve the lean adjoint ODE backwards:

$$\tilde{a}_{\tau-h} = \tilde{a}_\tau + h \tilde{a}_\tau^\top \nabla_{X_\tau} \left(2v^{\text{base}}(s, X_\tau, \tau) - \frac{1}{\tau} X_\tau \right). \quad (13)$$

 Stop gradient: $X_\tau \leftarrow \text{stopgrad}(X_\tau)$, $\tilde{a}_\tau \leftarrow \text{stopgrad}(\tilde{a}_\tau)$.

 Optimize θ w.r.t. the adjoint matching objective:

$$\mathcal{L}_{\text{Adj-Match}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{s \in \mathcal{B}} \sum_{\tau} \left\| \frac{2}{\sigma_n(\tau)} (v_\theta^{\text{ft}}(s, X_\tau, \tau) - v^{\text{base}}(s, X_\tau, \tau)) + \sigma_n(\tau) \tilde{a}_\tau \right\|^2 \quad (14)$$

Trust region update:

 Estimate path-space KL surrogate:

$$\hat{D}_n = \frac{1}{|\mathcal{B}|} \sum_{s \in \mathcal{B}} \sum_{\tau} \frac{2h}{g(\tau)^2} \|v_\theta^{\text{ft}}(s, X_\tau, \tau) - v^{\text{base}}(s, X_\tau, \tau)\|^2 \quad (15)$$

 EMA smoothing: $\overline{D}_n \leftarrow (1-\rho) \overline{D}_{n-1} + \rho \hat{D}_n$.

 Dual descent: $\lambda_{n+1} \leftarrow \max\{0, \lambda_n + \eta_\lambda (\overline{D}_n - \varepsilon_{\text{KL}})\}$.

end for

Output: fine-tuned velocity field v_θ^{ft} , critic Q_ϕ .

B Baselines

We compare against six baselines spanning distinct fine-tuning paradigms for flow-matching policies. Throughout, (s, a, r, s') are sampled uniformly from the replay buffer D without re-weighting; D contains the offline dataset during offline training and is augmented with online rollouts during online fine-tuning.

FQL [39]. FQL distills a multi-step flow policy into a one-step policy to avoid backpropagation through time. Conditioned on state s , the behavior-cloning rollout is given by the ODE

$$dX_\tau = v_\theta(X_\tau, \tau; s) d\tau, \quad X_0 \sim \mathcal{N}(0, I), \quad \tau \in [0, 1], \quad (16)$$

and we define $\text{ODE}(v_\theta, s, X_0) := X_1$ as its terminal value. The one-step policy $\pi_\omega(s, X_0)$ is trained jointly with v_θ to maximize the critic while staying close to this rollout:

$$\mathcal{L}_{\text{onestep}}(\omega) = \mathbb{E}_{X_0 \sim \mathcal{N}} \left[\underbrace{-Q(s, \pi_\omega(s, X_0))}_{\text{RL maximization}} + \underbrace{\alpha \|\pi_\omega(s, X_0) - \text{ODE}(v_\theta, s, X_0)\|_2^2}_{\text{BC distillation}} \right], \quad (17)$$

where α controls how closely π_ω stays to the BC rollout. The environment policy is π_ω .

CGQL-Linux. CGQL-Linux is a baseline introduced in Li and Levine [25], combining a BC velocity field with classifier-free guidance [8] from a Q -function. It trains an auxiliary intermediate critic $Q_\psi(s, X_\tau, \tau)$ on intermediate noisy actions X_τ , used to construct a guidance velocity that steers sampling toward the entropy-regularized optimal policy $\pi^*(\cdot | s) \propto e^{\beta Q_\phi(s, \cdot)}$. The guidance velocity is

$$\hat{v}_\psi(X_\tau, \tau; s) := \frac{(1 - \tau)\beta \nabla_{X_\tau} Q_\psi(s, X_\tau, \tau) + X_\tau}{\tau}, \quad (18)$$

where $X_\tau = (1 - \tau)X_0 + \tau X_1$ with $X_0 \sim \mathcal{N}(0, I)$. The final sampling velocity is $v = v^{\text{base}} + w \hat{v}_\psi$, where w modulates guidance strength. The intermediate critic Q_ψ is trained via a Linex regression [40, 33]:

$$\mathcal{L}_{\text{Linex}}(\psi) = \mathbb{E}_{\tau, X_0} [\exp(\beta(Q_\phi(s, X_1) - Q_\psi(s, X_\tau, \tau))) + \beta Q_\psi(s, X_\tau, \tau)], \quad (19)$$

while the standard critic Q_ϕ is trained via standard TD with target actions sampled from the full velocity v :

$$\mathcal{L}_{\text{TD}}(\phi) = \mathbb{E} \left[\left(Q_\phi(s, a) - r - \gamma Q_\phi(s', \text{ODE}(v, s', X_0)) \right)^2 \right]. \quad (20)$$

We follow Li and Levine [25] in using a Huber-style stabilization of the Linex loss to prevent exponential blow-up.

DSRL [51]. DSRL performs RL directly in the noise space of a frozen flow policy. It trains a one-step Gaussian noise-space policy $\pi_\omega(X_0 | s)$ via SAC, maximizing a noise-space critic $Q_\psi(s, X_0)$ that regresses to the original action-space critic:

$$L(\psi) = \mathbb{E}_{X_0 \sim \mathcal{N}} \left[\left(Q_\psi(s, X_0) - Q_\phi(s, \text{ODE}(v_\theta, s, X_0)) \right)^2 \right]. \quad (21)$$

At inference, actions are obtained by sampling $X_0 \sim \pi_\omega(\cdot | s)$ and pushing it through the flow policy: $a = \text{ODE}(v_\theta, s, X_0)$. Following Li and Levine [25], we modify the original DSRL to also fine-tune the BC velocity online (using a target network $v_{\bar{\theta}}$ for stability), which yields stronger offline-to-online performance.

IFQL. IFQL is the flow counterpart of implicit diffusion Q -learning [14], considered as a baseline in Park et al. [39]. Value learning uses IQL-style expectile regression [22]: a value network V_ξ is trained to fit an upper expectile of the critic, and the critic is bootstrapped through V_ξ :

$$\mathcal{L}_V(\xi) = \mathbb{E} [L_2^\kappa(Q_\phi(s, a) - V_\xi(s))], \quad (22)$$

$$\mathcal{L}_Q(\phi) = \mathbb{E} \left[\left(r + \gamma V_\xi(s') - Q_\phi(s, a) \right)^2 \right], \quad (23)$$

where $L_2^\kappa(u) = |\kappa - \mathbf{1}(u < 0)|u^2$ is the expectile loss with parameter $\kappa \in (0.5, 1)$. Policy extraction uses rejection sampling: N candidate actions are drawn from a BC flow policy by sampling $X_0^{(i)} \sim \mathcal{N}(0, I)$ and computing $a^{(i)} = \text{ODE}(v_\theta, s, X_0^{(i)})$ for $i = 1, \dots, N$, and the action with the highest Q_ϕ value is selected.

QAM and QAM-E [25]. QAM is the closest prior work to TRQAM: it solves a memoryless SOC problem analogous to (3) but *without* the $\sqrt{\lambda}$ scaling on the diffusion coefficient (i.e., the special case $\lambda = 1$ of our framework), with an inverse temperature β applied to the terminal reward $\beta Q_\phi(s, X_1)$. The fine-tuned velocity v_θ is trained against the BC velocity v^{base} through the lean adjoint matching loss

$$L_{\text{AM}}(\theta) = \mathbb{E} \left[\int_0^1 \left\| \frac{2(v_\theta(X_\tau, \tau; s) - v^{\text{base}}(X_\tau, \tau; s))}{\sigma(\tau)} + \sigma(\tau) \tilde{a}_\tau \right\|_2^2 d\tau \right], \quad (24)$$

where $\tilde{a}_\tau$ is the lean adjoint state with terminal condition $\tilde{a}_1 = -\beta \nabla_{X_1} Q_\phi(s, X_1)$, and X_τ follows the memoryless SDE in (2) with $\lambda = 1$. Element-wise gradient clipping is applied for numerical stability. QAM-E augments QAM with an additional residual edit policy $\pi_\omega(\Delta a \mid s, \tilde{a})$ that perturbs the QAM-generated action $\tilde{a}$ by at most σ_a in L_∞ distance (enforced by a tanh-squashed Gaussian), trained via entropy-regularized SAC with automatic entropy tuning. Both QAM and QAM-E share their BC velocity, critic, and inner adjoint solver with TRQAM; the only difference from TRQAM is that β is fixed rather than adapted via projected dual descent on λ .

C Experimental details

C.1 Domains and tasks

We evaluate on two benchmarks: OGBench [38] and Robomimic [32].

OGBench is a recent offline goal-conditioned RL benchmark. While OGBench originally designed for offline-goal-conditioned-RL, we use its reward based single-task variants. From OGBench, we use 10 domains spanning long-horizon navigation, multi-object manipulation, and combinatorial planning. Abbreviations: scene, puzzle-3x3 (p33), puzzle-4x4 (p44), cube-double (c2), cube-triple (c3), cube-quadruple (c4), humanoidmaze-medium (hm), humanoidmaze-large (hl), antmaze-large (al), and antmaze-giant (ag).

Robomimic is a demonstration based manipulation benchmark used as a stability stress-test. We use the lift, can, and square tasks.

The dataset size, episode length, and action dimension for each domain are reported in table 2. For each method and task, we run 8 random seeds. Unless otherwise stated, tables report mean success rate $\pm$ standard deviation across seeds, and plots show the mean with shaded regions denoting standard deviation.

Dataset structure. Unlike teleoperation-style benchmarks where each demonstration directly solves the target task, OGBench’s offline data is task-agnostic: navigate datasets capture free maze exploration and play datasets capture unstructured object manipulation. The BC-pretrained policy thus serves as a behavioral prior rather than a task-specific solution.

Dataset sources. We use the official OGBench datasets [38] for all domains except where noted below. For cube-triple-10M-* and puzzle-4x4-10M-*, we use a 10M-size subset of the official 100M release. The 100M release is split into 100 files of 1M transitions each, and we take the first 10 files sorted by name, following [21]. For antmaze-giant-10M-*, OGBench does not release a pretrained dataset at the size used in our experiments, so we generate it ourselves using the official OGBench data-generation pipeline with default settings. For Robomimic, we use the Multi-Human (MH) datasets, each consisting of 300 trajectories collected by six operators of varying proficiency (two “worse”, two “okay”, and two “better”), yielding diverse mixed-quality demonstrations [32].

C.2 Compute resources

Hardware. Experiments ran on an internal heterogeneous GPU cluster. The two dominant GPU types were NVIDIA GeForce RTX 2080 Ti (11 GB GPU memory) and NVIDIA A100-SXM4-80GB (80 GB GPU memory; ≈ 1.5 TB host memory and 64 logical CPU cores per node). A small fraction of runs (notably the scene OGBench domain and parts of antmaze-giant-10M) additionally used NVIDIA RTX 3090, RTX A6000, or RTX 4090 cards as availability allowed. To improve cluster

throughput, we packed up to four runs per A100-80GB GPU concurrently, while RTX 2080 Ti runs were single-tenant.

Per-run wallclock by hardware and scale. Median per-run wallclock times, measured directly from our experiment logs, are reported separately by GPU type because A100-80GB and RTX 2080 Ti are not interchangeable in run time:

- **Robomimic** (lift, can, square): RTX 2080 Ti single-tenant median ≈ 10 h on lift, ≈ 12 h on can, ≈ 12 h on square; A100-80GB with four-run multi-tenancy median ≈ 11 h on lift, ≈ 11 h on can, ≈ 12 h on square.
- **OGBench 1M-data domains** (antmaze-large, puzzle-3x3, scene, humanoidmaze-medium, humanoidmaze-large, cube-double, cube-triple): RTX 2080 Ti single-tenant median ≈ 7 h; the small fraction of these runs scheduled on A100 (four-run multi-tenancy) or RTX 3090 finished in ≈ 2 –4 h.
- **OGBench 10M-data domain** (antmaze-giant-10M, cube-triple-10M, puzzle-4x4-10M): RTX 2080 Ti single-tenant median ≈ 19 h; A100-80GB with four-run multi-tenancy median ≈ 9 h.
- **OGBench 100M-data domains** (cube-quadruple-100M): A100-80GB with four-run multi-tenancy median ≈ 7 h (this domain was run only on A100).

Total compute by category. Summing run wallclocks across all logged runs that contributed to the reported results, the two main reporting categories of the project are:

- **OGBench main reproduction and supporting ablations** across the 10 domains, 8 evaluation seeds, and the 7 compared methods (TRQAM, QAM, QAM-E, FQL, IFQL, CGQL-L, DSRL), including BC pretraining of the flow-matching priors: $\approx 38,400$ wallclock-run-hours, of which $\approx 30,700$ h on RTX 2080 Ti, $\approx 5,700$ h on A100-80GB, and $\approx 1,900$ h on RTX 3090, RTX A6000, and RTX 4090 cards combined.
- **Robomimic main comparison** on lift, can, and square for TRQAM, QAM, QAM-E (QAM-EDIT), and DSRL with the per-method hyperparameter sweeps in Appendix C.3: $\approx 8,600$ wallclock-run-hours, of which $\approx 6,000$ h on A100-80GB and $\approx 2,600$ h on RTX 2080 Ti.

The two categories together total approximately 47,000 wallclock-run-hours.

Physical GPU-hours. Accounting for the four-run multi-tenancy on A100-80GB and the single-tenant policy on the remaining GPU types, the wallclock-run-hours above translate to approximately 33,400 RTX 2080 Ti GPU-hours, 2,900 A100-80GB GPU-hours, and 1,900 GPU-hours on the RTX 3090/A6000/4090 cards combined—about 38,000 physical GPU-hours for the two reporting categories. Additional preliminary, failed, and supporting compute (KL-budget sweeps used to select ε_{KL} , sensitivity analyses in Appendix H.3, and prior re-pretraining) consumed further GPU-hours that are not included in the totals above.

C.3 Hyperparameters

Most methods share the common hyperparameters in table 3; method-specific hyperparameters are listed in table 4.

OGBench tuning. For all baselines (FQL, DSRL, IFQL, CGQL-L, QAM, QAM-E), we adopt the per-domain values reported as optimal in QAM [25], which were selected through an extensive per-domain hyperparameter sweep covering 6–20 configurations per method across all ten domains, totaling approximately 32,000 GPU-hours. For TRQAM, we tune ε_{KL} on two tuning tasks per domain with 2 seeds per configuration (different from the evaluation seeds), sweeping over $\{0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0\}$. Following Li and Levine [25], we use task 1 (the default task) and task 4 for locomotion domains, and task 2 (the default task) and task 4 for manipulation domains, as this combination better covers the characteristics of each domain than the default task alone. We select the configuration based on the combined offline-to-online learning curve and stability across the two tuning tasks; selected values are reported in table 4. For antmaze-giant-10M-*, we use a time-varying schedule: $\varepsilon_{\text{KL}} = 0.5$ during offline training and 3.0 during online fine-tuning. The offline value is selected from the per-domain sweep, where it yielded the most stable offline training. However, retaining 0.5 online led to slow improvement in success rate, reflecting the larger

exploration demands of giant-scale navigation. We therefore relax the bound at the online transition, selecting 3.0 from the same sweep range as a sweet spot between exploration and stability (see section H.2 for a direct comparison of static vs. relaxed schedules). This schedule exploits TRQAM’s capability to track time-varying ε_{KL} without retraining. All main-paper results are averaged over 8 evaluation seeds.

Robomimic tuning. Since Robomimic was not evaluated in the QAM paper [25], no per-domain hyperparameters are available, so we conducted an independent sweep for each method (DSRL, QAM, QAM-E, TRQAM) on this benchmark. As Robomimic serves as our stability testbed, we run the full sweep range with 8 seeds per configuration to obtain reliable estimates of variance across hyperparameters. For QAM-E, which has two hyperparameters, we sweep the full Cartesian product ($4 \times 2 = 8$ configurations). Robomimic consists of human teleoperation data, for which staying closer to behavior cloning is generally beneficial. We therefore adjust the QAM [25] sweep ranges toward configurations that more strongly anchor the policy to the behavior policy. The sweep ranges are listed in table 5.

Table 2: Domain metadata.

Domain	Data	Horizon	Act. dim.
cube-double-*	1M	500	5
cube-triple-10M-*	10M	1000	5
cube-quadruple-100M-*	100M	1000	5
antmaze-large-*	1M	1000	8
antmaze-giant-10M-*	10M	1000	8
humanoidmaze-medium-*	1M	2000	21
humanoidmaze-large-*	1M	2000	21
scene-*	1M	750	5
puzzle-3x3-*	1M	500	5
puzzle-4x4-10M-*	10M	500	5
lift	31127	500	5
can	62756	500	5
square	80731	500	5

Table 3: Common hyperparameters.

Parameter	Value
Batch size	256
Discount factor (γ)	0.995 (default), 0.999 (humanoidmaze)
Optimizer	Adam
Learning rate	3×10^{-4}
Target network update rate	5×10^{-3}
Critic ensemble size (K)	10
Critic pessimism coefficient (ρ)	0.5 (default), 0 (humanoidmaze)
UTD ratio	1
Number of flow steps (T)	10
BC training steps	0.3×10^6
Offline RL steps	10^6
Online RL steps	0.5×10^6
Network width	512 (default), 1024 (10M/100M data)
Network depth	4 hidden layers
Gradient max-norm clipping	False (default), 1 (QAM, QAM-E, TRQAM)
Actor layer norm	False (default), True (10M/100M data)
Critic layer norm	True

Table 4: **Domain-specific hyperparameters.**

Domain	FQL α	DSRL σ_z	IFQL κ	CGQL-L $(\vartheta, \varrho, \tau)$	QAM β	QAM-E (β, σ_a)	TRQAM ε_{KL}
scene-*	300	0.4	0.9	(10, 0.1, 0.1)	1	(1, 0)	0.5
puzzle-3x3-*	300	1.0	0.95	(10, 0.001, 0.1)	3	(1, 0.1)	2.0
puzzle-4x4-10M-*	1	1.0	0.9	(10, 0.001, 1)	30	(0.1, 0.9)	4.0
cube-double-*	300	1.0	0.9	(10, 0.001, 0.01)	1	(1, 0)	0.5
cube-triple-10M-*	30	1.4	0.95	(10, 0.001, 0.1)	3	(3, 0.1)	0.5
cube-quadruple-100M-*	100	1.4	0.95	(10, 0.1, 0.01)	1	(3, 0.1)	1.0
antmaze-large-*	3	0.8	0.9	(10, 0.001, 0.1)	10	(1, 0.1)	1.0
antmaze-giant-10M-*	3	1.2	0.8	(10, 0.001, 0.1)	3	(10, 0.1)	0.5 $\rightarrow$ 3.0
humanoidmaze-medium-*	30	0.6	0.7	(10, 0.1, 0.1)	3	(3, 0.1)	0.5
humanoidmaze-large-*	30	0.8	0.8	(10, 0.1, 0.1)	3	(3, 0.1)	0.5
lift	-	0.8	-	-	1	(1, 0.03)	0.1
can	-	0.6	-	-	0.3	(1, 0.03)	0.1
square	-	0.8	-	-	0.1	(1, 0.03)	0.5

Table 5: **Robomimic hyperparameter sweep ranges.**

Method	Hyperparameter(s)	Sweep Range
DSRL	σ_z	{0.03, 0.1, 0.2, 0.4, 0.6, 0.8}
QAM	β	{0.01, 0.03, 0.1, 0.3, 1, 3}
QAM-E	(β, σ_a)	{(0.03, 0.1, 0.3, 1)}, {(0.03, 0.1)}
TRQAM	ε_{KL}	{0.01, 0.03, 0.1, 0.5, 1.0, 1.5}

D Proofs

This section provides full proofs for the three theoretical results stated in Section 3 of the main text: Theorem 1 (path-space KL identity), Proposition 1 (Terminal KL upper-bounded by path-space KL), and Lemma 1 (exponential amplification of critic errors).

We work throughout on a filtered probability space $(\Omega, \mathcal{F}, (\mathcal{F}_\tau)_{\tau \in [0,1]}, \mathbb{P})$ equipped with a natural filtration and an initial state distribution $X_0 \sim p_0$. We adopt standard regularity assumptions for controlled diffusions, as in Nüsken and Richter [36]: the coefficients b and σ are sufficiently smooth, b has at most linear growth, $\sigma\sigma^\top$ is uniformly positive definite, and the admissible control set $\mathcal{U}$ consists of progressively measurable controls with at most linear growth. Crucially, to ensure strong duality in our KL-budgeted improvement problem, we assume that the set of achievable path measures $\{\mathbb{P}^u \mid u \in \mathcal{U}\}$ is convex. Moreover, we assume that $u \in \mathcal{U}$ satisfies Novikov’s condition,

$$\mathbb{E}_{\mathbf{X} \sim \mathbb{P}^{\text{base}}} \left[\exp \left(\frac{1}{2\lambda} \int_0^1 \|u(X_\tau, \tau)\|^2 d\tau \right) \right] < \infty,$$

which justifies the application of Girsanov’s theorem in the proof of Theorem 1.

D.1 Proof of Lemma 1: exponential amplification of critic errors

Lemma 1 (Exponential amplification of critic errors). *Fix $s \in \mathcal{S}$ and let $Q, \tilde{Q} : \mathcal{A} \rightarrow \mathbb{R}$ satisfy $\|Q - \tilde{Q}\|_\infty \leq \varepsilon$. Assume $\pi_{\text{base}}(\cdot \mid s) > 0$ a.e. and define*

$$\pi_Q(a \mid s) = \frac{\pi_{\text{base}}(a \mid s) e^{\beta Q(a)}}{Z_Q}, \quad \pi_{\tilde{Q}}(a \mid s) = \frac{\pi_{\text{base}}(a \mid s) e^{\beta \tilde{Q}(a)}}{Z_{\tilde{Q}}},$$

with normalizers $Z_Q, Z_{\tilde{Q}} \in (0, \infty)$. Then

$$\text{TV}(\pi_Q, \pi_{\tilde{Q}}) \leq \frac{1}{2}(e^{2\beta\varepsilon} - 1), \quad D_{\text{KL}}(\pi_Q \parallel \pi_{\tilde{Q}}) \leq 2\beta\varepsilon.$$

Proof. For notational simplicity, we suppress s throughout the proof. From $\|Q - \tilde{Q}\|_\infty \leq \varepsilon$, multiplying $-\varepsilon \leq Q(a) - \tilde{Q}(a) \leq \varepsilon$ by β and exponentiating yields, for all a ,

$$e^{-\beta\varepsilon} \leq e^{\beta(Q(a) - \tilde{Q}(a))} \leq e^{\beta\varepsilon}. \quad (25)$$

Multiplying (25) by $\pi_{\text{base}}(a) e^{\beta\tilde{Q}(a)} \geq 0$ and integrating gives $e^{-\beta\varepsilon} Z_{\tilde{Q}} \leq Z_Q \leq e^{\beta\varepsilon} Z_{\tilde{Q}}$. Combining this with (25) in the likelihood ratio

$$\frac{\pi_Q(a)}{\pi_{\tilde{Q}}(a)} = e^{\beta(Q(a) - \tilde{Q}(a))} \cdot \frac{Z_{\tilde{Q}}}{Z_Q}$$

shows that both factors lie in $[e^{-\beta\varepsilon}, e^{\beta\varepsilon}]$, so

$$e^{-2\beta\varepsilon} \leq \frac{\pi_Q(a)}{\pi_{\tilde{Q}}(a)} \leq e^{2\beta\varepsilon} \quad \text{a.e.} \quad (26)$$

The TV bound follows from (26): $|\pi_Q(a) - \pi_{\tilde{Q}}(a)| \leq (e^{2\beta\varepsilon} - 1) \pi_{\tilde{Q}}(a)$ a.e., and integrating with $\int \pi_{\tilde{Q}} = 1$ gives $\text{TV}(\pi_Q, \pi_{\tilde{Q}}) = \frac{1}{2} \|\pi_Q - \pi_{\tilde{Q}}\|_1 \leq \frac{1}{2} (e^{2\beta\varepsilon} - 1)$. The KL bound also follows from (26): $\log(\pi_Q/\pi_{\tilde{Q}}) \leq 2\beta\varepsilon$ a.e., so

$$D_{\text{KL}}(\pi_Q \parallel \pi_{\tilde{Q}}) = \int \pi_Q(a) \log \frac{\pi_Q(a)}{\pi_{\tilde{Q}}(a)} da \leq 2\beta\varepsilon \int \pi_Q(a) da = 2\beta\varepsilon.$$

□

D.2 Proof of Theorem 1: path-space KL identity

Theorem 1 (Path-space KL identity with explicit λ dependence). *Let $\mathbb{P}^u$ and $\mathbb{P}^{\text{base}}$ denote the path measures on $C([0, 1]; \mathbb{R}^d)$ induced by the base and controlled SDEs, respectively:*

$$\begin{aligned} dX_\tau^{\text{base}} &= b(X_\tau^{\text{base}}, \tau) d\tau + \sqrt{\lambda} \sigma(\tau) dB_\tau^{\text{base}}, \\ dX_\tau^u &= (b(X_\tau^u, \tau) + \sigma(\tau) u(X_\tau^u, \tau)) d\tau + \sqrt{\lambda} \sigma(\tau) dB_\tau^u, \end{aligned}$$

with common initial distribution $X_0 \sim p_0$. Then

$$D_{\text{KL}}(\mathbb{P}^u(\mathbf{X} \mid X_0) \parallel \mathbb{P}^{\text{base}}(\mathbf{X} \mid X_0)) = \mathbb{E}_{\mathbf{X} \sim \mathbb{P}^u} \left[\frac{1}{2\lambda} \int_0^1 \|u(X_\tau^u, \tau)\|^2 d\tau \right].$$

Proof. Let $(\mathcal{F}_\tau)_{\tau \in [0, 1]}$ denote the natural filtration on the canonical path space $C([0, 1]; \mathbb{R}^d)$. By Girsanov's theorem [37, Theorem 8.6.6], the Radon–Nikodym derivative of $\mathbb{P}^u$ with respect to $\mathbb{P}^{\text{base}}$ restricted to $\mathcal{F}_\tau$ is

$$Z_\tau := \frac{d\mathbb{P}^u}{d\mathbb{P}^{\text{base}}} \Big|_{\mathcal{F}_\tau} = \exp \left(\frac{1}{\sqrt{\lambda}} \int_0^\tau u(X_s^u, s)^\top dB_s^{\text{base}} - \frac{1}{2\lambda} \int_0^\tau \|u(X_s^u, s)\|^2 ds \right), \quad (27)$$

where B^{base} is a standard Brownian motion under $\mathbb{P}^{\text{base}}$. By the definition of KL divergence on the full path space (terminal σ -algebra $\mathcal{F}_1$),

$$D_{\text{KL}}(\mathbb{P}^u \parallel \mathbb{P}^{\text{base}}) = \mathbb{E}_{\mathbf{X} \sim \mathbb{P}^u} [\log Z_1].$$

Next, define

$$B_\tau^u := B_\tau^{\text{base}} - \frac{1}{\sqrt{\lambda}} \int_0^\tau u(X_s^u, s) ds.$$

By Girsanov's theorem, B^u is a standard Brownian motion under $\mathbb{P}^u$; in differential form, $dB_s^{\text{base}} = dB_s^u + \frac{1}{\sqrt{\lambda}} u(X_s^u, s) ds$. Substituting this into (27) at $\tau = 1$ gives

$$\begin{aligned} \log Z_1 &= \frac{1}{\sqrt{\lambda}} \int_0^1 u(X_s^u, s)^\top \left(dB_s^u + \frac{1}{\sqrt{\lambda}} u(X_s^u, s) ds \right) - \frac{1}{2\lambda} \int_0^1 \|u(X_s^u, s)\|^2 ds \\ &= \frac{1}{\sqrt{\lambda}} \int_0^1 u(X_s^u, s)^\top dB_s^u + \frac{1}{2\lambda} \int_0^1 \|u(X_s^u, s)\|^2 ds. \end{aligned}$$

Taking expectation under $\mathbb{P}^u$ and using that the Itô integral $\int_0^1 u(X_s^u, s)^\top dB_s^u$ is a zero-mean martingale yields

$$D_{\text{KL}}(\mathbb{P}^u \parallel \mathbb{P}^{\text{base}}) = \mathbb{E}_{\mathbf{X} \sim \mathbb{P}^u} \left[\frac{1}{2\lambda} \int_0^1 \|u(X_s^u, s)\|^2 ds \right],$$

which is the claimed identity. $\square$

Comparison to prior SOC parameterizations. The SOC parameterization used in prior work [10] takes the form

$$dX_\tau^u = (b(X_\tau^u, \tau) + \sigma(\tau) u(X_\tau^u, \tau)) d\tau + \sigma(\tau) dB_\tau^u$$

without a $\sqrt{\lambda}$ scaling on the diffusion coefficient. Then Girsanov theorem yields

$$D_{\text{KL}}(\mathbb{P}^u \parallel \mathbb{P}^{\text{base}}) = \mathbb{E}_{\mathbf{X} \sim \mathbb{P}^u} \left[\frac{1}{2} \int_0^1 \|u(X_s^u, s)\|^2 ds \right],$$

In this form, the path-space KL coincides with the SOC quadratic cost up to a fixed constant; there is no parameter λ inside the SDE that modulates the strength of this regularization, so adapting the trust-region strength would require an external coefficient applied to the SOC objective itself. By scaling the diffusion by $\sqrt{\lambda}$ instead, our parameterization makes λ an intrinsic parameter of the controlled dynamics, with the $1/\lambda$ factor appearing directly in the KL identity above. This is the structural property that allows λ to serve as an adaptive dual variable for KL-budgeted improvement (Section 3): adjusting λ reshapes the SDE itself, which in turn directly modulates the path-space KL.

D.3 Proof of Proposition 1: Terminal KL upper-bounded by path-space KL

Proposition 1 (Terminal KL upper-bounded by path-space KL). *Let $\mathbb{P}^u$ and $\mathbb{P}^{\text{base}}$ denote the path measures on $C([0, 1]; \mathbb{R}^d)$ induced by the base and controlled SDEs, respectively:*

$$\begin{aligned} dX_\tau^{\text{base}} &= b(X_\tau^{\text{base}}, \tau) d\tau + \sqrt{\lambda} \sigma(\tau) dB_\tau^{\text{base}}, \\ dX_\tau^u &= (b(X_\tau^u, \tau) + \sigma(\tau) u(X_\tau^u, \tau)) d\tau + \sqrt{\lambda} \sigma(\tau) dB_\tau^u, \end{aligned}$$

with common initial distribution $X_0 \sim p_0$. Let $\pi_\theta(\cdot | s)$ and $\pi_{\text{base}}(\cdot | s)$ denote the corresponding terminal action distributions at $\tau = 1$, respectively. Then

$$D_{\text{KL}}(\pi_\theta(\cdot | s) \parallel \pi_{\text{base}}(\cdot | s)) \leq D_{\text{KL}}(\mathbb{P}^u(\mathbf{X} | X_0) \parallel \mathbb{P}^{\text{base}}(\mathbf{X} | X_0)), \quad (28)$$

Proof. Let $\Pi : C([0, 1]; \mathbb{R}^d) \rightarrow \mathbb{R}^d$ denote the deterministic terminal projection $\Pi(\mathbf{X}) = X_1$. By construction, the terminal action distributions are pushforwards of the path measures: $\pi_\theta(\cdot | s) = \Pi_\# \mathbb{P}^u$ and $\pi_{\text{base}}(\cdot | s) = \Pi_\# \mathbb{P}^{\text{base}}$. Applying the data-processing inequality for KL divergence under maps [42, Corollary. 2.18] to Π yields

$$D_{\text{KL}}(\Pi_\# \mathbb{P}^u \parallel \Pi_\# \mathbb{P}^{\text{base}}) \leq D_{\text{KL}}(\mathbb{P}^u \parallel \mathbb{P}^{\text{base}}),$$

which proves the claim. $\square$

E Path-space KL surrogate under OT memoryless discretization

This section derives the closed-form path-space KL estimator $\hat{D}_n$ used in Algorithm 3. Recall from Theorem 1 that the path-space KL between $\mathbb{P}^u$ and $\mathbb{P}^{\text{base}}$ admits a closed-form expression in terms of the control u , and from Proposition 1 that this path-space KL upper-bounds the terminal-policy KL we ultimately wish to constrain. We thus seek a tractable estimator of the path-space KL under the discretized OT memoryless sampler.

Gaussian KL with shared covariance. For two Gaussians $\mathcal{N}(\mu_1, \Sigma)$ and $\mathcal{N}(\mu_0, \Sigma)$ with shared covariance $\Sigma \succ 0$, specializing the multivariate Gaussian KL formula [42, Equation. 2.8] makes the log-determinant and trace terms vanish. In other words,

$$D_{\text{KL}}(\mathcal{N}(\mu_1, \Sigma) \parallel \mathcal{N}(\mu_0, \Sigma)) = \frac{1}{2} (\mu_1 - \mu_0)^\top \Sigma^{-1} (\mu_1 - \mu_0). \quad (29)$$

OT memoryless Euler step KL. Under the OT memoryless Euler scheme with step size h and schedule $g(\tau) = \sqrt{2(1-\tau)/\tau}$, the fine-tuned and base transitions at time τ_k given $X_{\tau_k} = x$ are both Gaussian with covariance $\Sigma_k = h g(\tau_k)^2 I$ and means

$$\mu_\theta(x, \tau_k) = x + h \left(2v_\theta^{\text{fin}}(x, \tau_k) - \frac{1}{\tau_k} x \right), \quad \mu_{\text{base}}(x, \tau_k) = x + h \left(2v^{\text{base}}(x, \tau_k) - \frac{1}{\tau_k} x \right).$$

The means differ only through the velocity fields: $\mu_\theta(x, \tau_k) - \mu_{\text{base}}(x, \tau_k) = 2h(v_\theta^{\text{fin}}(x, \tau_k) - v^{\text{base}}(x, \tau_k))$. Substituting into (29) with $\Sigma_k^{-1} = \frac{1}{h g(\tau_k)^2} I$ gives the per-step KL

$$D_{\text{KL}}(p_\theta(\cdot | x) \| p_{\text{base}}(\cdot | x)) = \frac{2h}{g(\tau_k)^2} \|v_\theta^{\text{fin}}(x, \tau_k) - v^{\text{base}}(x, \tau_k)\|^2. \quad (30)$$

Chain rule and Monte Carlo estimator. Let $\mathbb{P}^u$ and $\mathbb{P}^{\text{base}}$ now denote the discrete-time path measures of the Markov chains $\mathbf{X} = (X_\tau)_{\tau \in \{0, h, \dots, 1\}}$ with shared initial distribution p_0 and the Gaussian transition kernels above. Under mild regularity conditions, the Markov KL chain rule gives

$$D_{\text{KL}}(\mathbb{P}^u \| \mathbb{P}^{\text{base}}) = \sum_{\tau \in \{0, h, \dots, 1-h\}} \mathbb{E}_{X_\tau \sim \mathbb{P}^u} [D_{\text{KL}}(p_\theta(\cdot | X_\tau) \| p_{\text{base}}(\cdot | X_\tau))]. \quad (31)$$

Substituting (30) and estimating the expectation by a Monte Carlo average over the trajectories in batch $\mathcal{B}$ yields

$$\hat{D} = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{X} \in \mathcal{B}} \sum_{\tau \in \{0, h, \dots, 1-h\}} \frac{2h}{g(\tau)^2} \|v_\theta^{\text{fin}}(X_\tau, \tau) - v^{\text{base}}(X_\tau, \tau)\|^2, \quad (32)$$

which is the estimator used in Algorithm 3 (optionally smoothed by EMA to reduce variance).

F KL-budgeted improvement: primal–dual derivation

This section derives the projected dual update on λ used in Algorithm 3.

Primal problem and Slater’s condition. We consider the path-space KL-budgeted improvement problem

$$\max_{u \in \mathcal{U}} \mathbb{E}_{X \sim \mathbb{P}^u} [Q^\pi(X_1)] \quad (33)$$

$$\text{s.t. } D_{\text{KL}}(\mathbb{P}^u \| \mathbb{P}^{\text{base}}) \leq \varepsilon_{\text{KL}}. \quad (34)$$

Viewed in the path measure space, the objective is linear in $\mathbb{P}^u$ and $D_{\text{KL}}(\cdot \| \mathbb{P}^{\text{base}})$ is convex; the trivial control $u \equiv 0$ recovers $\mathbb{P}^u = \mathbb{P}^{\text{base}}$ and is strictly feasible whenever $\varepsilon_{\text{KL}} > 0$, so Slater’s condition holds and strong duality applies.

Lagrangian and dual function. Let $\lambda \geq 0$ be the dual variable for the KL constraint. The Lagrangian is

$$\mathcal{L}(u, \lambda) = \mathbb{E}_{X \sim \mathbb{P}^u} [Q^\pi(X_1)] + \lambda(\varepsilon_{\text{KL}} - D_{\text{KL}}(\mathbb{P}^u \| \mathbb{P}^{\text{base}})),$$

and the dual function is $g(\lambda) := \sup_u \mathcal{L}(u, \lambda)$. By strong duality, minimizing g over $\lambda \geq 0$ solves the primal. A standard subgradient of g at λ is

$$s(\lambda) = \varepsilon_{\text{KL}} - D_{\text{KL}}(\mathbb{P}^{u_\lambda} \| \mathbb{P}^{\text{base}}), \quad (35)$$

where $u_\lambda := \arg \sup_u \mathcal{L}(u, \lambda)$ denotes the inner maximizer at the current dual variable. The subgradient inequality follows immediately: for any $\lambda' \geq 0$,

$$g(\lambda') \geq \mathcal{L}(u_\lambda, \lambda') = g(\lambda) + (\lambda' - \lambda)(\varepsilon_{\text{KL}} - D_{\text{KL}}(\mathbb{P}^{u_\lambda} \| \mathbb{P}^{\text{base}})).$$

Projected dual descent. Projected subgradient descent on $\min_{\lambda \geq 0} g(\lambda)$ would require evaluating the KL at the inner maximizer u_λ , which is not available in closed form. In Algorithm 3, we instead evaluate the path-space KL at the control u_n at iteration n via the Monte Carlo surrogate $\hat{D}_n$ from (32). The dual update

$$\lambda_{n+1} \leftarrow \max\{0, \lambda_n + \eta_\lambda(\bar{D}_n - \varepsilon_{\text{KL}})\} \quad (36)$$

preserves the key sign property: when the realized KL exceeds the budget, λ rises and the controlled dynamics become more conservative; when it falls below, λ relaxes the trust region.

G Internal vs. external KL regularization: detailed comparison

Section 3.3 contrasts two ways of pairing the dual update (9) with a KL constraint: appending $\bar{D}_n$ as an auxiliary loss term (*external*), or letting λ scale the diffusion coefficient inside the SOC dynamics (*internal*, TRQAM). The two formulations share the same dual update rule and the same KL estimator, yet they differ in how the constraint is enforced. This appendix expands on that contrast and explains why the difference is structural rather than cosmetic.

The key asymmetry is in what λ represents. In the external formulation, λ is a scalar weight on an auxiliary KL regularization term in the loss: it balances this regularization against the critic signal, but does not appear in the controlled SDE itself. The path-space KL between $\mathbb{P}^u$ and $\mathbb{P}^{\text{base}}$ is therefore independent of λ , and is determined entirely by the learned control u . Adapting λ only changes the relative weight between the adjoint-matching loss $\mathcal{L}_{\text{Adj-Match}}$ and the KL regularization $\bar{D}_n$ during gradient descent; whether the realized KL ends up close to ε_{KL} depends on the optimizer balancing these competing signals. Under strong reward gradients, the KL term can be effectively overridden, and the realized KL drifts above the target KL bound ε_{KL} .

In the internal formulation, by contrast, λ is a parameter of the controlled SDE itself. Theorem 1 establishes that the path-space KL is exactly $\mathbb{E}[\frac{1}{2\lambda} \int \|u\|^2]$, so λ appears as the inverse coefficient of the KL itself. Increasing λ shrinks the path-space KL directly through the SDE’s diffusion term, rather than indirectly through the critic gradient. The dual update therefore reshapes the trajectory distribution structurally rather than competing with critic guidance. Table 6 summarizes these distinctions.

Table 6: **Internal vs. external KL regularization.** Although both approaches can use the same dual update rule on λ , the role of λ differs structurally. Internalizing λ inside the SDE makes the path-space KL an exact function of λ via Girsanov (Theorem 1), turning the target KL bound ε_{KL} into a structural constraint rather than a soft penalty.

	External KL (auxiliary regularization)	Internal KL (TRQAM)
Controlled SDE	$dX_\tau^u = (b + \sigma u) d\tau + \sigma(\tau) dB_\tau$	$dX_\tau^u = (b + \sigma u) d\tau + \sqrt{\lambda} \sigma(\tau) dB_\tau$
Path-space KL	$\mathbb{E}\left[\frac{1}{2} \int_0^1 \ u\ ^2 d\tau\right]$ (λ -independent)	$\mathbb{E}\left[\frac{1}{2\lambda} \int_0^1 \ u\ ^2 d\tau\right]$ (Theorem 1)
Role of λ	Regularization weight on auxiliary loss	Intrinsic SDE parameter
Training loss	$\mathcal{L}_{\text{Adj-Match}}(\theta) + \lambda \bar{D}_n(\theta)$	$\mathcal{L}_{\text{Adj-Match}}(\theta)$
Effect of increasing λ	Competes with reward gradient at loss level	Reshapes the SDE at sampling level
Enforcement mechanism	Soft (gradient competition)	Structural (via dynamics)
Realized KL vs. target bound	Exceed the target bound (Figure 14, 15)	Tracks target bound tightly

A useful way to see the practical consequence is that the two approaches differ in *when* the KL constraint takes effect. External regularization acts *after* trajectories are generated: the SDE produces samples freely under the current u , and the KL term enters only at the loss level, where it competes with the reward gradient during optimization. The realized path-space KL has no direct tie to λ ; whether it ends up close to ε_{KL} depends on the optimizer balancing these competing signals. Internal regularization, by contrast, enforces the constraint *during* sample generation. Because $\sqrt{\lambda} \sigma(\tau) = \sqrt{2(1-\tau)}/\tau$ is fixed by the OT schedule, adjusting λ co-adjusts $\sigma(\tau)$ and reshapes the entire controlled SDE, including its drift term $b(x, \tau) + \sigma(\tau) u(x, \tau)$: increasing λ shrinks $\sigma(\tau)$, which weakens the control contribution $\sigma(\tau) u(x, \tau)$ and pulls the controlled SDE toward the base dynamics. By Theorem 1, the realized path-space KL is then an exact function of λ , so the dual update directly reshapes the trajectory distribution rather than competing with critic guidance at the loss level. This distinction is what makes ε_{KL} a structural constraint in TRQAM rather than a nominal target. Appendix Figures 14 and 15 demonstrate this empirically. On Robomimic lift and Robomimic can, external regularization lets the realized KL exceed the prescribed bound across all six target values, with corresponding degradation in success rate.

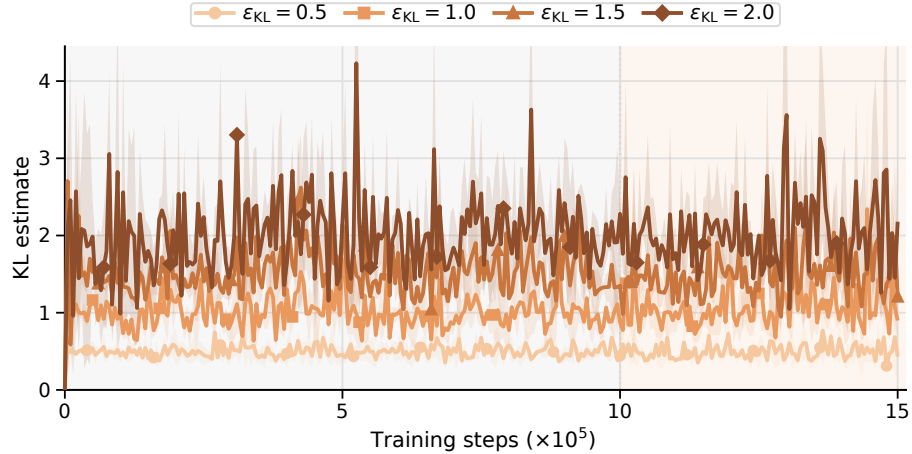


Figure 7: **TRQAM tracks the prescribed KL budget across offline and online training.** We vary the target KL bound $\varepsilon_{\text{KL}} \in \{0.5, 1.0, 1.5, 2.0\}$ and plot the realized path-space KL throughout training. Larger budgets produce correspondingly larger policy deviation, and the monotonic ordering is preserved across the offline-to-online transition.

H Additional experiments

H.1 TRQAM tightly enforces the prescribed KL budget

TRQAM faithfully enforces the prescribed KL budget across a range of target values. Figure 7 plots the realized path-space KL under four budgets $\varepsilon_{\text{KL}} \in \{0.5, 1.0, 1.5, 2.0\}$ throughout offline and online training. The realized KL tracks each prescribed target with a clear monotonic ordering, and this ordering is preserved across the offline-to-online transition, indicating that the dual update remains effective under distribution shift. The KL constraint in TRQAM is therefore both controllable and stable across training regimes.

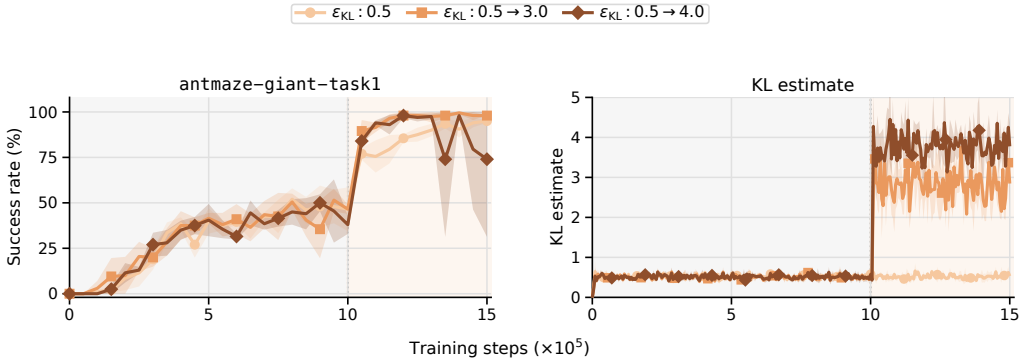


Figure 8: **TRQAM tracks time-varying ε_{KL} schedules across the offline-to-online transition.** We illustrate this on `antmaze-giant`, the domain in our benchmark with the largest state space and accordingly the highest exploration demands during online fine-tuning. We keep $\varepsilon_{\text{KL}} = 0.5$ during offline training and optionally switch to a larger bound at the online transition (vertical dashed line). *Right:* The realized KL adapts to each new target almost immediately, showing that the dual update on λ handles schedule changes without instability. *Left:* The static schedule (0.5) improves more slowly online than the moderate schedule (0.5 $\rightarrow$ 3.0), which reaches $\sim 98\%$ success rapidly and outperforms the more aggressive (0.5 $\rightarrow$ 4.0) alternative.

H.2 Time-varying KL budget

The KL bound ε_{KL} is not necessarily a fixed quantity throughout training: in offline-to-online RL, the appropriate trust region may differ between phases, since online fine-tuning often requires more exploration than the offline phase permits. TRQAM accommodates this naturally because λ is internalized in the SOC dynamics rather than entering as a soft loss penalty: the dual update on λ remains stable under time-varying ε_{KL} , allowing the bound to be re-set at any point without retraining or instability.

We illustrate this on `antmaze-giant`, the domain in our benchmark with the largest state space and accordingly the highest exploration demands during online fine-tuning. Figure 8 sweeps three schedules on `antmaze-giant-task1`, the tuning task for our per-domain ε_{KL} choice: the static schedule (0.5, matching the offline value) and two relaxations at the online transition ($0.5 \rightarrow 3.0$, $0.5 \rightarrow 4.0$). Two observations follow.

First, the realized KL adapts to each new target almost immediately after the switch, confirming that TRQAM tracks time-varying bounds without loss of stability.

Second, the static schedule (0.5) improves substantially more slowly online than the moderate schedule ($0.5 \rightarrow 3.0$). On domains with large state spaces such as `antmaze-giant`, online fine-tuning relies on exploring beyond the offline support, and a tight KL budget restricts this exploration and slows adaptation to the new state distribution. The moderate schedule reaches $\sim 98\%$ success rapidly, while the more aggressive $0.5 \rightarrow 4.0$ introduces substantial variance, identifying 3.0 as a sweet spot.

H.3 Sensitivity Analysis

We extend the four-task sweep of Section 4.3 to all ten OGBench domains, sweeping $\varepsilon_{\text{KL}} \in \{0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0\}$ on the default tuning task of each domain with all other hyperparameters fixed. Results across 8 seeds appear in Figure 9, showing two patterns. Tight budgets are best on `humanoidmaze-medium`, `humanoidmaze-large`, `cube-double`, `cube-triple`, and `cube-quadruple`. Larger budgets monotonically improve performance on `puzzle-4x4`, in keeping with its notably larger state space.

Smaller ε_{KL} also produces slower online adaptation across the sweep, consistent with the dual update tightening the trust region. We observe the same effect on `antmaze-giant` (Section H.2), where the static 0.5 schedule slows online adaptation in a domain whose large state space demands broader exploration. Combined with the realized-KL ordering of Figure 5, this is consistent with ε_{KL} controlling realized deviation from π_{base} rather than acting as a nominal target. Because TRQAM tightly enforces the chosen budget (Section 4.2), ε_{KL} remains a hyperparameter, but picking it for a new task comes down to task structure.

H.4 Stability stress test on Robomimic

We complement our OGBench results with Robomimic [32], a standard manipulation benchmark on which we observe fixed-temperature adjoint matching to be unstable.

Fixed-temperature methods collapse across the sweep. Sweeping QAM over six values of $\beta \in \{0.01, 0.03, 0.1, 0.3, 1.0, 3.0\}$ and QAM-E over eight (β, σ_a) configurations, the collapse pattern persists across most settings on `lift` and `can` (See Figure 10, 11), the adjoint-matching loss explodes during offline training and success rate collapses to near zero. This is consistent with the structural fragility predicted by fixed- β critic-error amplification (Lemma 1).

External KL is insufficient; TRQAM enforces the budget. External KL regularization partially mitigates the collapse, but the realized path-space KL substantially exceeds the prescribed bound across every budget, leaving the policy vulnerable to critic-error amplification. TRQAM, by contrast, tracks each budget tightly throughout offline training and across the offline-to-online transition, and remains stable across the full sweep $\varepsilon_{\text{KL}} \in \{0.01, 0.03, 0.1, 0.5, 1, 1.5\}$. (See Figure 14, 15, 16).

I Broader impacts

TRQAM is a methodological contribution to stable off-policy fine-tuning of pretrained flow-matching policies, evaluated entirely on simulated benchmarks (OGBench, Robomimic). On the positive side, more reliable fine-tuning of pretrained robot policies can reduce the data and compute required to specialize useful behaviors and lessen the brittleness of pretrained-prior degradation under TD bootstrapping, which is a common source of instability in real-world RL deployments. On the negative side, the same trust-region machinery makes off-policy improvement of expressive flow policies more reliable, which could in principle accelerate the deployment of autonomous systems whose downstream uses we cannot fully anticipate; in particular, when applied to settings beyond the simulated benchmarks studied here, fine-tuned policies could exhibit failure modes (e.g., physical safety violations in robotics) that an algorithmic KL bound does not directly address. We view standard mitigations—safety filters at deployment, sandboxed evaluation, and explicit reward and constraint design—as necessary complements rather than substitutes for the algorithmic guarantees provided by TRQAM. The paper does not release pretrained models or datasets that pose dual-use or high-risk concerns.

Table 7: Full offline results at 1M training steps (8 seeds).

		FQL	CGQL-L	DSRL	IFQL	QAM	QAM-E	TRQAM
antmaze-large	task1	24 $\pm$ 44	62 $\pm$ 38	45 $\pm$ 12	7 $\pm$ 11	<u>96</u> $\pm$ 3	93 $\pm$ 5	<u>95</u> $\pm$ 4
	task2	0 $\pm$ 0	21 $\pm$ 36	73 $\pm$ 7	9 $\pm$ 14	36 $\pm$ 49	<u>91</u> $\pm$ 4	<u>87</u> $\pm$ 6
	task3	73 $\pm$ 14	88 $\pm$ 5	84 $\pm$ 5	62 $\pm$ 24	86 $\pm$ 7	<u>93</u> $\pm$ 3	<u>93</u> $\pm$ 3
	task4	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0	35 $\pm$ 22	0 $\pm$ 0	66 $\pm$ 9	<u>75</u> $\pm$ 17
	task5	92 $\pm$ 4	68 $\pm$ 42	63 $\pm$ 8	34 $\pm$ 24	<u>94</u> $\pm$ 3	89 $\pm$ 5	<u>96</u> $\pm$ 3
	agg. (5 tasks)	38 $\pm$ 9	48 $\pm$ 7	53 $\pm$ 2	29 $\pm$ 8	62 $\pm$ 9	86 $\pm$ 3	<u>89</u> $\pm$ 4
antmaze-giant-10M	task1	1 $\pm$ 1	11 $\pm$ 12	2 $\pm$ 3	0 $\pm$ 0	37 $\pm$ 9	0 $\pm$ 0	<u>48</u> $\pm$ 5
	task2	0 $\pm$ 0	14 $\pm$ 12	0 $\pm$ 0	<u>43</u> $\pm$ 13	18 $\pm$ 19	0 $\pm$ 0	21 $\pm$ 12
	task3	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0	<u>14</u> $\pm$ 8	2 $\pm$ 2	0 $\pm$ 0	5 $\pm$ 5
	task4	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0	1 $\pm$ 2	20 $\pm$ 24	0 $\pm$ 0	<u>58</u> $\pm$ 13
	task5	10 $\pm$ 29	8 $\pm$ 23	2 $\pm$ 2	2 $\pm$ 2	69 $\pm$ 8	28 $\pm$ 38	<u>75</u> $\pm$ 7
	agg. (5 tasks)	2 $\pm$ 6	7 $\pm$ 5	1 $\pm$ 1	12 $\pm$ 3	29 $\pm$ 4	6 $\pm$ 8	<u>41</u> $\pm$ 4
humanoidmaze-medium	task1	84 $\pm$ 21	84 $\pm$ 9	24 $\pm$ 23	<u>93</u> $\pm$ 3	33 $\pm$ 34	28 $\pm$ 30	<u>87</u> $\pm$ 5
	task2	<u>99</u> $\pm$ 1	<u>99</u> $\pm$ 1	88 $\pm$ 6	94 $\pm$ 4	<u>100</u> $\pm$ 1	<u>98</u> $\pm$ 2	89 $\pm$ 7
	task3	89 $\pm$ 9	0 $\pm$ 0	64 $\pm$ 27	<u>96</u> $\pm$ 4	90 $\pm$ 8	74 $\pm$ 18	<u>90</u> $\pm$ 6
	task4	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0	<u>82</u> $\pm$ 8	0 $\pm$ 0	0 $\pm$ 0	61 $\pm$ 7
	task5	<u>99</u> $\pm$ 1	<u>100</u> $\pm$ 1	88 $\pm$ 5	<u>99</u> $\pm$ 2	<u>100</u> $\pm$ 1	<u>99</u> $\pm$ 1	<u>95</u> $\pm$ 4
	agg. (5 tasks)	74 $\pm$ 5	57 $\pm$ 2	53 $\pm$ 10	<u>93</u> $\pm$ 2	64 $\pm$ 7	60 $\pm$ 6	84 $\pm$ 3
humanoidmaze-large	task1	1 $\pm$ 2	16 $\pm$ 11	1 $\pm$ 1	32 $\pm$ 5	6 $\pm$ 10	9 $\pm$ 14	<u>45</u> $\pm$ 8
	task2	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0	6 $\pm$ 9	0 $\pm$ 0	0 $\pm$ 0	<u>10</u> $\pm$ 6
	task3	7 $\pm$ 4	16 $\pm$ 8	2 $\pm$ 3	<u>76</u> $\pm$ 6	7 $\pm$ 5	3 $\pm$ 4	34 $\pm$ 15
	task4	0 $\pm$ 0	0 $\pm$ 1	0 $\pm$ 0	35 $\pm$ 26	0 $\pm$ 0	0 $\pm$ 0	<u>44</u> $\pm$ 13
	task5	0 $\pm$ 1	0 $\pm$ 0	1 $\pm$ 1	0 $\pm$ 0	5 $\pm$ 11	9 $\pm$ 18	<u>45</u> $\pm$ 6
	agg. (5 tasks)	2 $\pm$ 1	6 $\pm$ 3	1 $\pm$ 1	30 $\pm$ 7	4 $\pm$ 3	4 $\pm$ 5	<u>36</u> $\pm$ 4
scene	task1	<u>100</u> $\pm$ 0	<u>100</u> $\pm$ 0	<u>100</u> $\pm$ 0	<u>99</u> $\pm$ 1	<u>100</u> $\pm$ 0	<u>100</u> $\pm$ 0	<u>100</u> $\pm$ 0
	task2	<u>100</u> $\pm$ 1	<u>99</u> $\pm$ 1	<u>100</u> $\pm$ 0	2 $\pm$ 2	<u>100</u> $\pm$ 0	<u>100</u> $\pm$ 0	<u>100</u> $\pm$ 0
	task3	82 $\pm$ 5	93 $\pm$ 7	<u>100</u> $\pm$ 1	77 $\pm$ 7	94 $\pm$ 4	93 $\pm$ 4	<u>100</u> $\pm$ 1
	task4	60 $\pm$ 22	0 $\pm$ 0	<u>100</u> $\pm$ 0	2 $\pm$ 2	26 $\pm$ 21	22 $\pm$ 30	<u>93</u> $\pm$ 6
	task5	8 $\pm$ 7	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 1
	agg. (5 tasks)	70 $\pm$ 5	58 $\pm$ 1	<u>80</u> $\pm$ 0	36 $\pm$ 1	64 $\pm$ 4	63 $\pm$ 6	<u>79</u> $\pm$ 1
puzzle-3x3	task1	87 $\pm$ 11	0 $\pm$ 0	<u>100</u> $\pm$ 0	<u>100</u> $\pm$ 1	75 $\pm$ 15	99 $\pm$ 2	<u>100</u> $\pm$ 0
	task2	37 $\pm$ 39	0 $\pm$ 0	<u>100</u> $\pm$ 0	23 $\pm$ 7	0 $\pm$ 0	<u>100</u> $\pm$ 0	<u>100</u> $\pm$ 0
	task3	0 $\pm$ 0	0 $\pm$ 0	<u>100</u> $\pm$ 0	82 $\pm$ 11	0 $\pm$ 0	76 $\pm$ 11	<u>100</u> $\pm$ 0
	task4	0 $\pm$ 0	0 $\pm$ 0	<u>100</u> $\pm$ 0	29 $\pm$ 8	0 $\pm$ 0	87 $\pm$ 6	<u>100</u> $\pm$ 1
	task5	3 $\pm$ 6	0 $\pm$ 0	<u>100</u> $\pm$ 0	86 $\pm$ 9	0 $\pm$ 1	84 $\pm$ 20	<u>100</u> $\pm$ 0
	agg. (5 tasks)	25 $\pm$ 10	0 $\pm$ 0	<u>100</u> $\pm$ 0	64 $\pm$ 4	15 $\pm$ 3	89 $\pm$ 4	<u>100</u> $\pm$ 0
puzzle-4x4-10M	task1	3 $\pm$ 3	1 $\pm$ 1	90 $\pm$ 5	74 $\pm$ 11	3 $\pm$ 4	90 $\pm$ 10	<u>100</u> $\pm$ 0
	task2	6 $\pm$ 9	0 $\pm$ 0	9 $\pm$ 9	4 $\pm$ 3	1 $\pm$ 1	43 $\pm$ 22	<u>99</u> $\pm$ 1
	task3	23 $\pm$ 15	0 $\pm$ 0	81 $\pm$ 20	84 $\pm$ 6	2 $\pm$ 3	57 $\pm$ 10	<u>98</u> $\pm$ 2
	task4	7 $\pm$ 5	0 $\pm$ 0	76 $\pm$ 28	46 $\pm$ 13	0 $\pm$ 1	31 $\pm$ 16	<u>100</u> $\pm$ 1
	task5	9 $\pm$ 16	0 $\pm$ 0	49 $\pm$ 43	3 $\pm$ 1	1 $\pm$ 1	50 $\pm$ 22	<u>98</u> $\pm$ 5
	agg. (5 tasks)	9 $\pm$ 7	0 $\pm$ 0	61 $\pm$ 8	42 $\pm$ 4	1 $\pm$ 1	54 $\pm$ 8	<u>99</u> $\pm$ 1
cube-double	task1	75 $\pm$ 10	83 $\pm$ 4	85 $\pm$ 7	18 $\pm$ 7	96 $\pm$ 3	94 $\pm$ 4	<u>98</u> $\pm$ 2
	task2	59 $\pm$ 11	66 $\pm$ 6	85 $\pm$ 5	9 $\pm$ 2	81 $\pm$ 6	82 $\pm$ 7	<u>92</u> $\pm$ 3
	task3	39 $\pm$ 8	72 $\pm$ 7	<u>82</u> $\pm$ 10	8 $\pm$ 4	75 $\pm$ 7	67 $\pm$ 8	<u>80</u> $\pm$ 12
	task4	6 $\pm$ 3	15 $\pm$ 5	35 $\pm$ 6	2 $\pm$ 2	24 $\pm$ 6	29 $\pm$ 6	<u>54</u> $\pm$ 8
	task5	44 $\pm$ 9	37 $\pm$ 6	71 $\pm$ 4	7 $\pm$ 3	78 $\pm$ 6	<u>82</u> $\pm$ 6	<u>82</u> $\pm$ 6
	agg. (5 tasks)	44 $\pm$ 4	55 $\pm$ 2	72 $\pm$ 4	9 $\pm$ 2	71 $\pm$ 2	71 $\pm$ 3	<u>81</u> $\pm$ 3
cube-triple-10M	task1	26 $\pm$ 20	2 $\pm$ 3	70 $\pm$ 18	51 $\pm$ 22	56 $\pm$ 22	26 $\pm$ 11	<u>87</u> $\pm$ 8
	task2	1 $\pm$ 1	0 $\pm$ 0	16 $\pm$ 7	16 $\pm$ 3	15 $\pm$ 11	14 $\pm$ 9	<u>42</u> $\pm$ 9
	task3	8 $\pm$ 6	0 $\pm$ 0	40 $\pm$ 8	28 $\pm$ 8	21 $\pm$ 11	9 $\pm$ 6	<u>45</u> $\pm$ 9
	task4	2 $\pm$ 3	0 $\pm$ 0	15 $\pm$ 5	1 $\pm$ 1	4 $\pm$ 3	3 $\pm$ 3	<u>30</u> $\pm$ 13
	task5	1 $\pm$ 1	0 $\pm$ 0	31 $\pm$ 12	23 $\pm$ 8	1 $\pm$ 1	2 $\pm$ 4	<u>48</u> $\pm$ 17
	agg. (5 tasks)	7 $\pm$ 5	0 $\pm$ 1	34 $\pm$ 6	24 $\pm$ 7	19 $\pm$ 6	11 $\pm$ 4	<u>50</u> $\pm$ 5
cube-quadruple-100M	task1	36 $\pm$ 25	7 $\pm$ 5	36 $\pm$ 12	15 $\pm$ 11	<u>70</u> $\pm$ 10	45 $\pm$ 19	<u>66</u> $\pm$ 16
	task2	4 $\pm$ 5	0 $\pm$ 1	5 $\pm$ 3	5 $\pm$ 7	2 $\pm$ 4	0 $\pm$ 0	<u>18</u> $\pm$ 11
	task3	3 $\pm$ 3	0 $\pm$ 0	2 $\pm$ 2	6 $\pm$ 8	<u>16</u> $\pm$ 11	2 $\pm$ 4	<u>11</u> $\pm$ 7
	task4	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 1	0 $\pm$ 0	<u>2</u> $\pm$ 2	0 $\pm$ 0	<u>3</u> $\pm$ 2
	task5	0 $\pm$ 0	0 $\pm$ 0	1 $\pm$ 1	<u>2</u> $\pm$ 3	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0
	agg. (5 tasks)	9 $\pm$ 5	1 $\pm$ 1	9 $\pm$ 3	6 $\pm$ 3	<u>18</u> $\pm$ 3	9 $\pm$ 3	<u>19</u> $\pm$ 5
all	agg. (50 tasks)	28	23	46	35	35	45	<u>68</u>

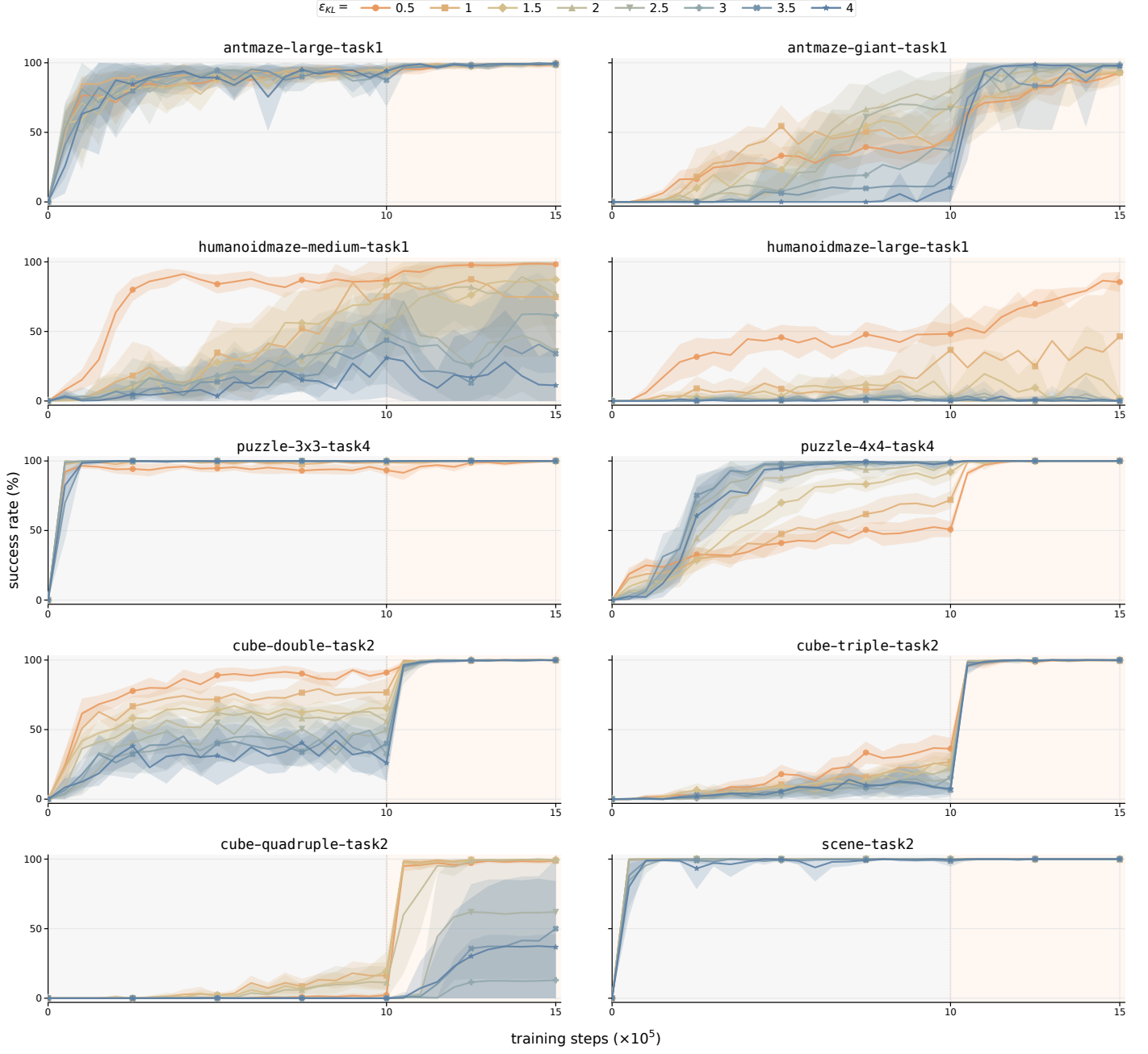


Figure 9: **Hyperparameter ε_{KL} sweep on OGBench [38] (8 seeds).** We sweep $\varepsilon_{KL} \in \{0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0\}$ on the default tuning task of each domain, with all other hyperparameters fixed to the main-comparison setting. Each panel reports success rate over training steps, with the offline-to-online transition at 10^6 steps. Shaded regions denote ± 1 standard deviation across seeds; evaluated tasks are listed in the panel titles. Two patterns emerge. Tight budgets are best on humanoidmaze-medium, humanoidmaze-large, cube-double, cube-triple, and cube-quadruple, while larger budgets monotonically improve performance on puzzle-4x4, consistent with its notably larger state space. Smaller ε_{KL} also produces slower online adaptation across the sweep, consistent with the dual update tightening the trust region.

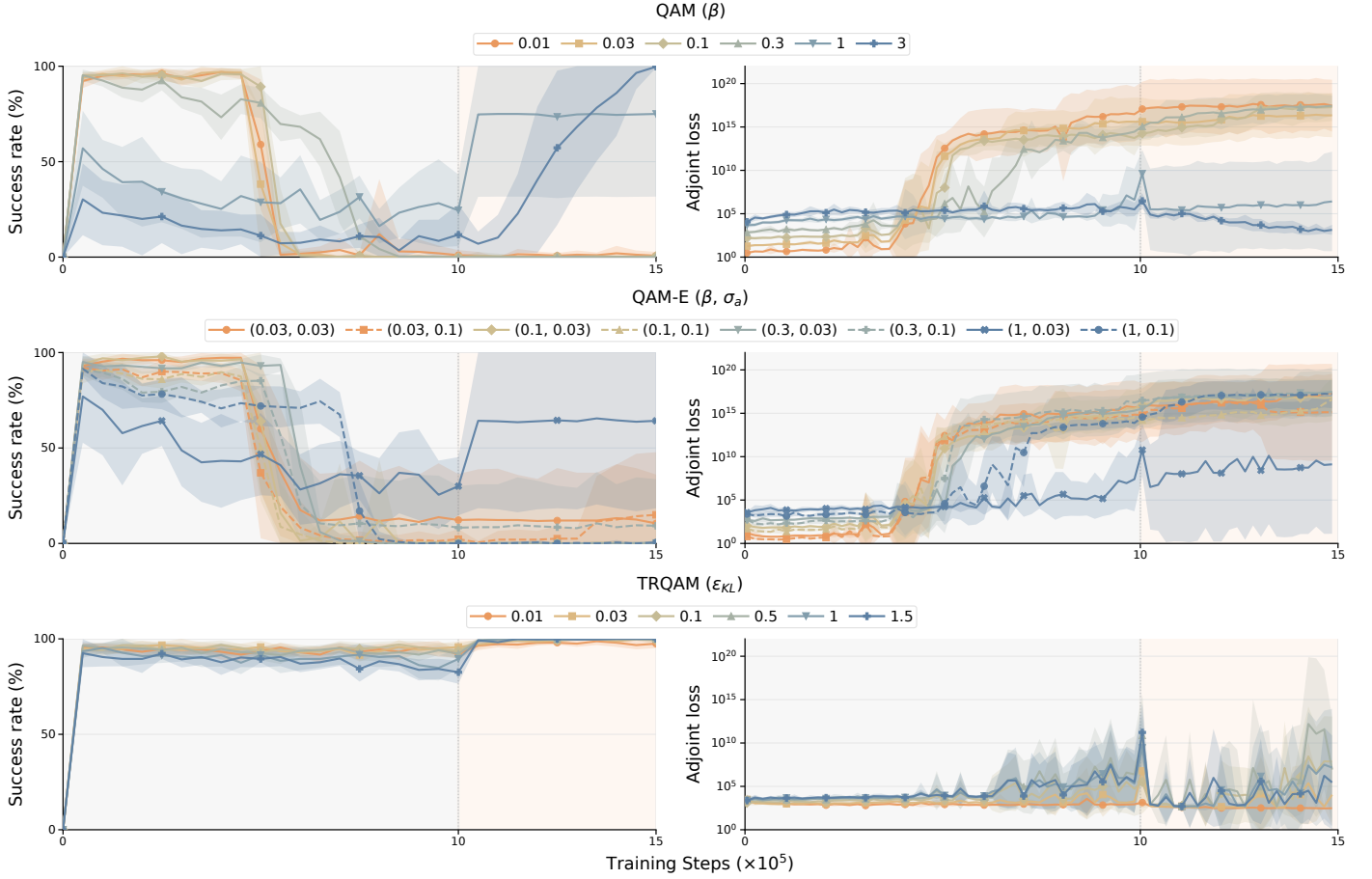


Figure 10: **Hyperparameter sweep on Robomimic-lift (8 seeds).** Sweep ranges for QAM, QAM-E, and TRQAM follow table 5. Left column reports success rate; right column reports adjoint-matching loss on log scale. Fixed-temperature methods (QAM, QAM-E) exhibit adjoint loss growth of 10 to 20 orders of magnitude across most settings, with corresponding success rate collapse. TRQAM remains stable across all six budgets. Shaded regions denote ± 1 standard deviation across seeds.

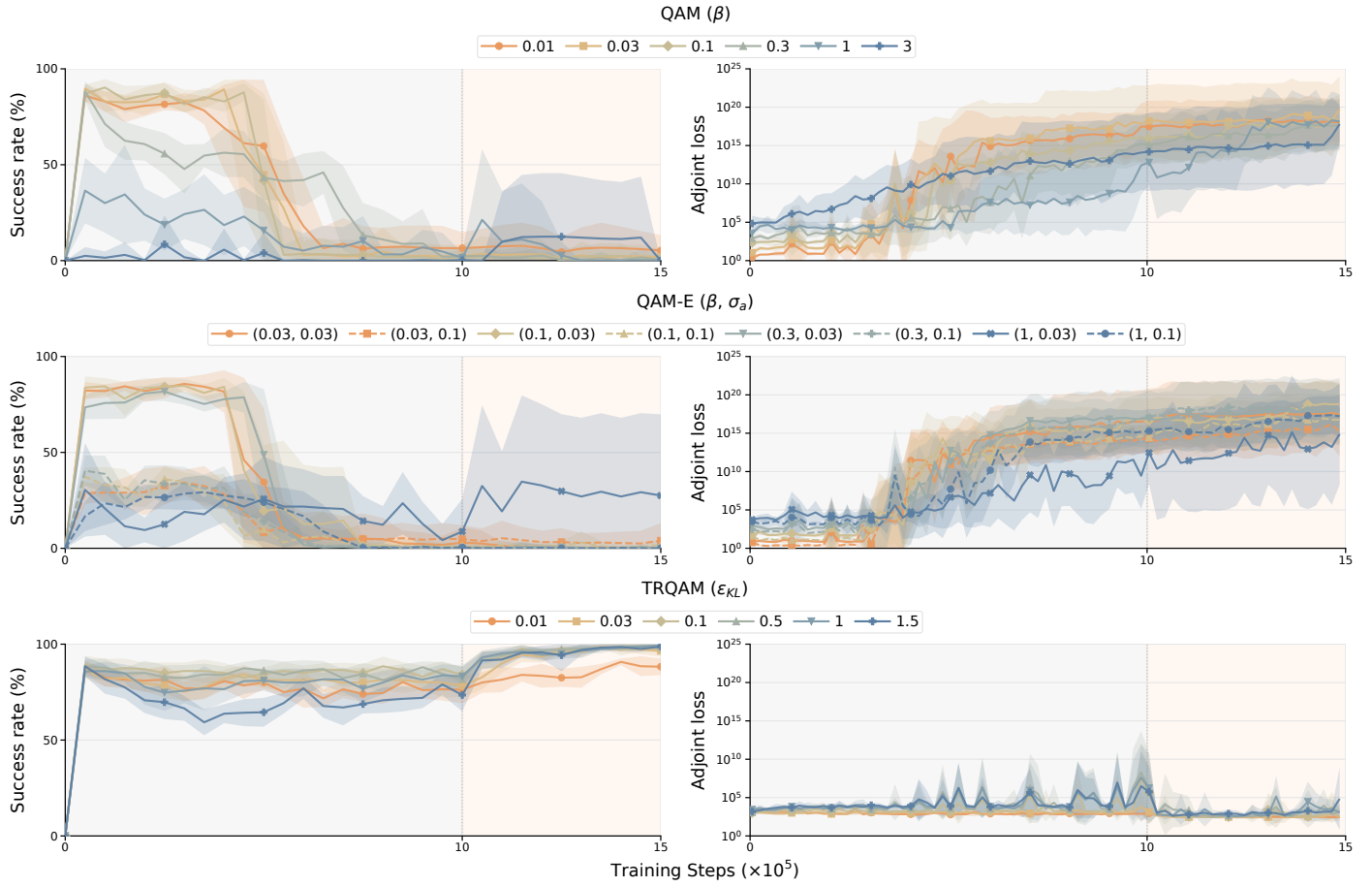


Figure 11: **Hyperparameter sweep on Robomimic-can (8 seeds).** Same setup as fig. 10. Fixed-temperature collapse is again hyperparameter-wide: QAM and QAM-E exhibit adjoint loss growth of 10 to 25 orders of magnitude across most settings with success rate collapse, while TRQAM remains stable across all six budgets. Shaded regions denote ± 1 standard deviation across seeds.

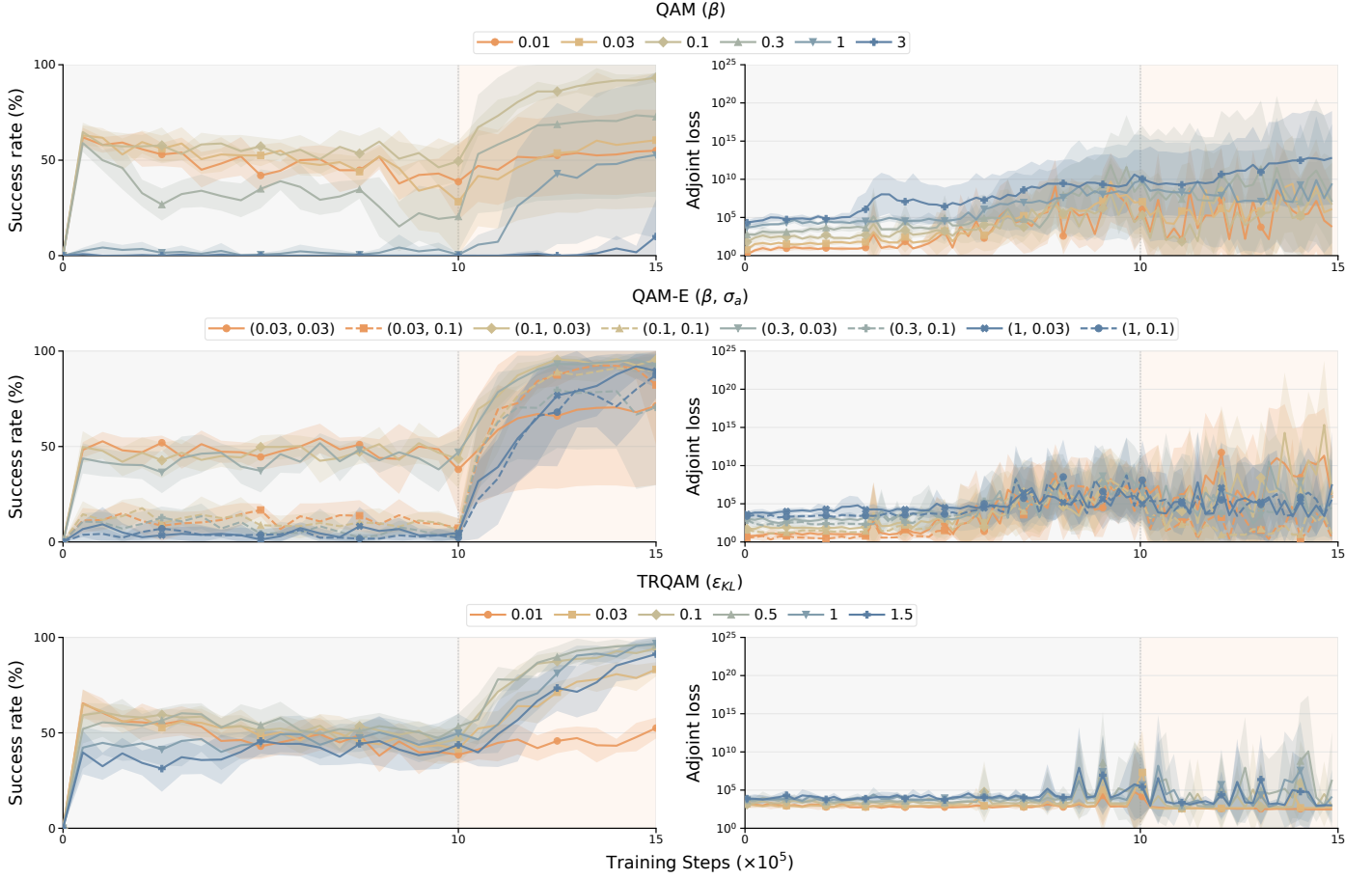


Figure 12: **Hyperparameter sweep on Robomimic-square (8 seeds).** Same setup as fig. 10. While the adjoint loss explosion is less severe than on *lift* and *can*, fixed-temperature variants still exhibit signs of instability across the sweep, with adjoint loss steadily growing during offline training. TRQAM, in contrast, maintains a bounded adjoint loss and matches or exceeds the fixed-temperature variants across the sweep. Shaded regions denote ± 1 standard deviation across seeds.

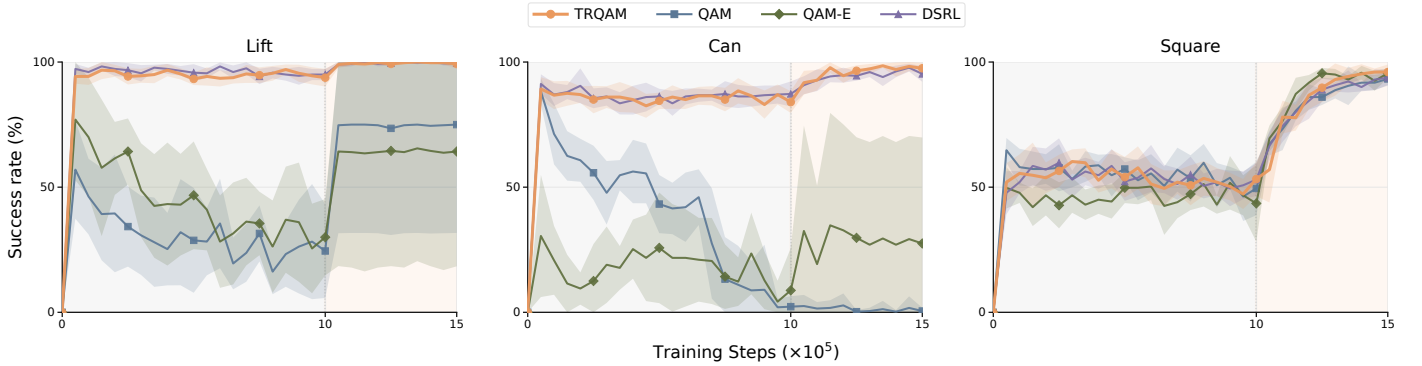


Figure 13: **Best per-method hyperparameters on Robomimic across all three tasks (8 seeds).** For each method (TRQAM, QAM, QAM-E, DSRL) we select the configuration with the strongest overall offline-to-online learning curve from the sweep in table 5. Shaded regions denote ± 1 standard deviation across seeds.

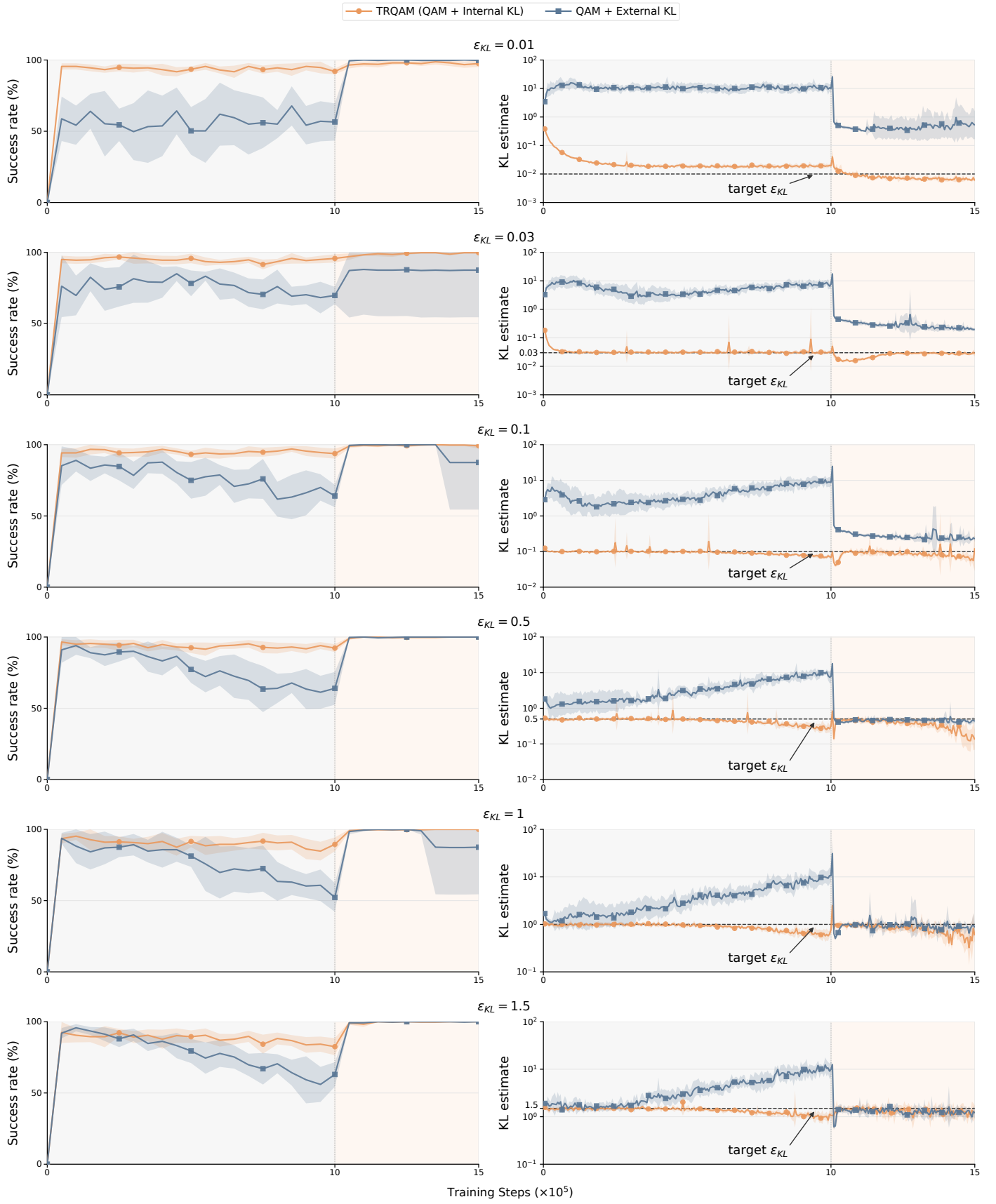


Figure 14: **Internal vs. external KL regularization on Robomimic-lift across all budgets (8 seeds).** Each row plots one target KL budget $\epsilon_{KL} \in \{0.01, 0.03, 0.1, 0.5, 1.0, 1.5\}$. *Left*: success rate over training steps; *right*: realized path-space KL with target shown as a dashed line. TRQAM (orange) tracks each prescribed budget tightly across offline and online training, whereas QAM with external KL regularization (blue) lets the realized KL drift well over it, with corresponding success rate degradation. Shaded regions denote ± 1 standard deviation across seeds.

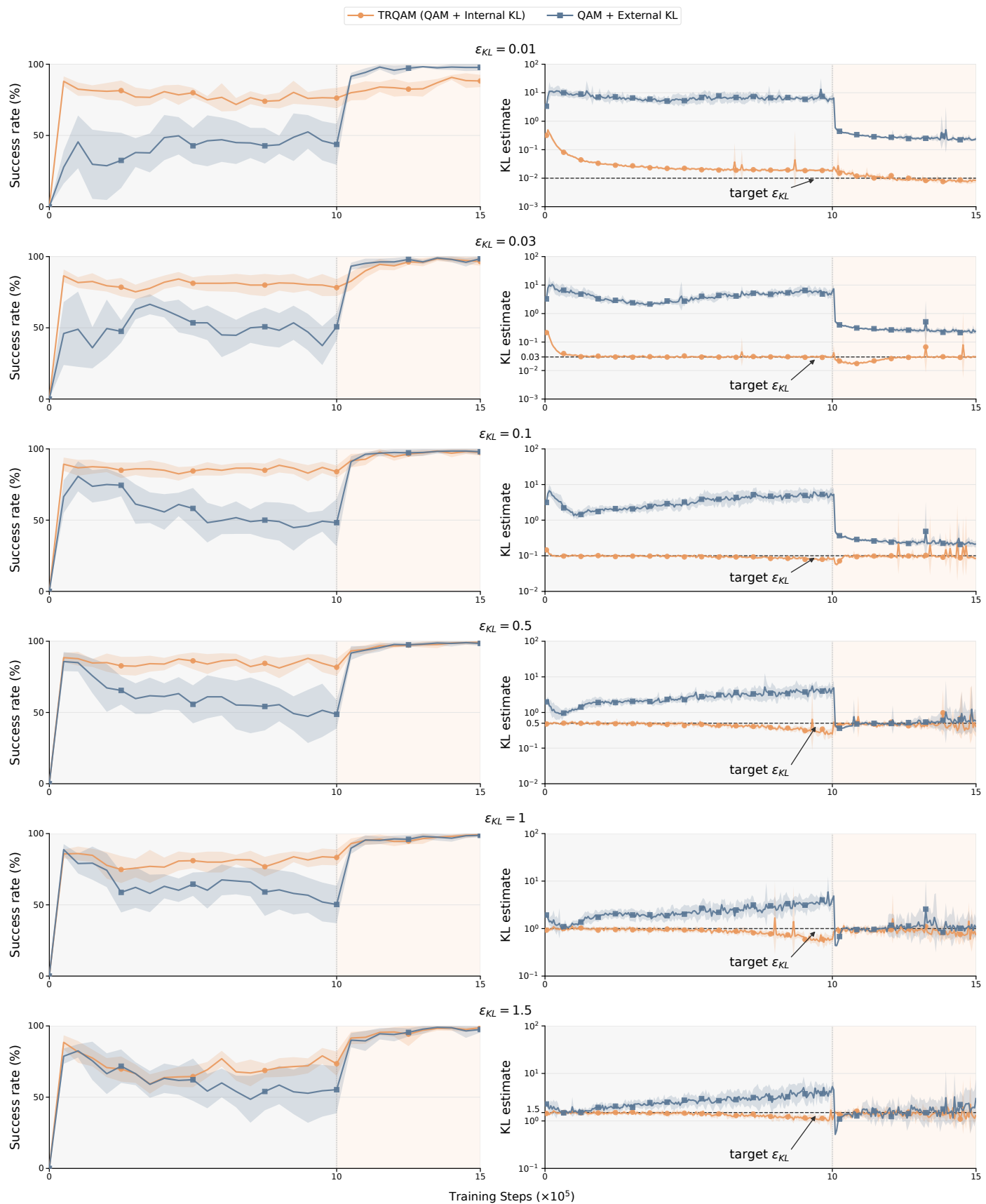


Figure 15: **Internal vs. external KL regularization on Robomimic-can across all budgets (8 seeds).** Each row plots one target KL budget $\epsilon_{KL} \in \{0.01, 0.03, 0.1, 0.5, 1.0, 1.5\}$. *Left:* success rate over training steps; *right:* realized path-space KL with target shown as a dashed line. TRQAM (orange) tracks each prescribed budget tightly across offline and online training, whereas QAM with external KL regularization (blue) lets the realized KL drift well over it, with corresponding success rate degradation. Shaded regions denote ± 1 standard deviation across seeds.

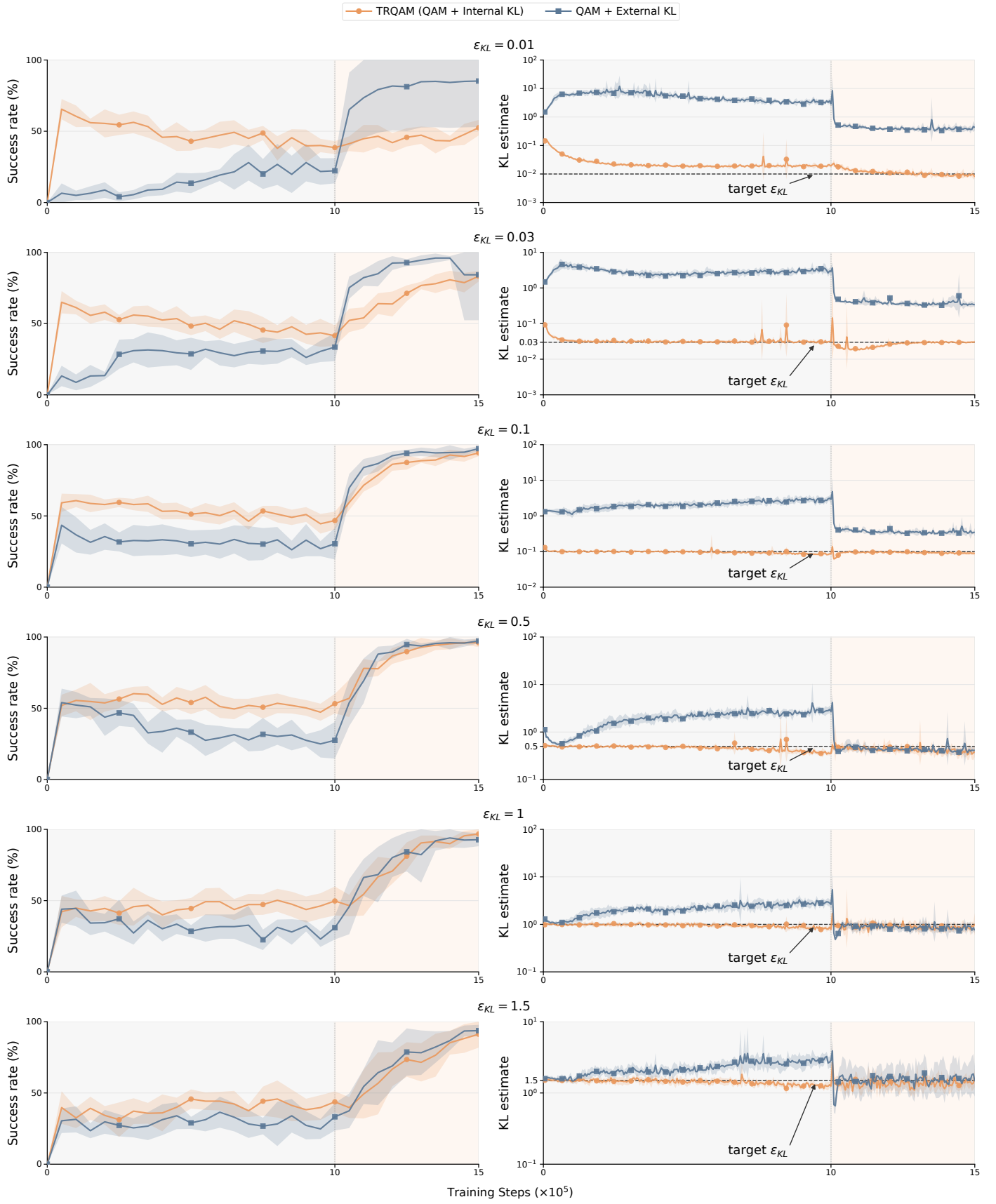


Figure 16: **Internal vs. external KL regularization on Robomimic-square across all budgets (8 seeds).** Each row plots one target KL budget $\epsilon_{KL} \in \{0.01, 0.03, 0.1, 0.5, 1.0, 1.5\}$. *Left*: success rate over training steps; *right*: realized path-space KL with target shown as a dashed line. TRQAM (orange) tracks each prescribed budget tightly across offline and online training, whereas QAM with external KL regularization (blue) lets the realized KL drift well over it, with corresponding success rate degradation. Shaded regions denote ± 1 standard deviation across seeds.

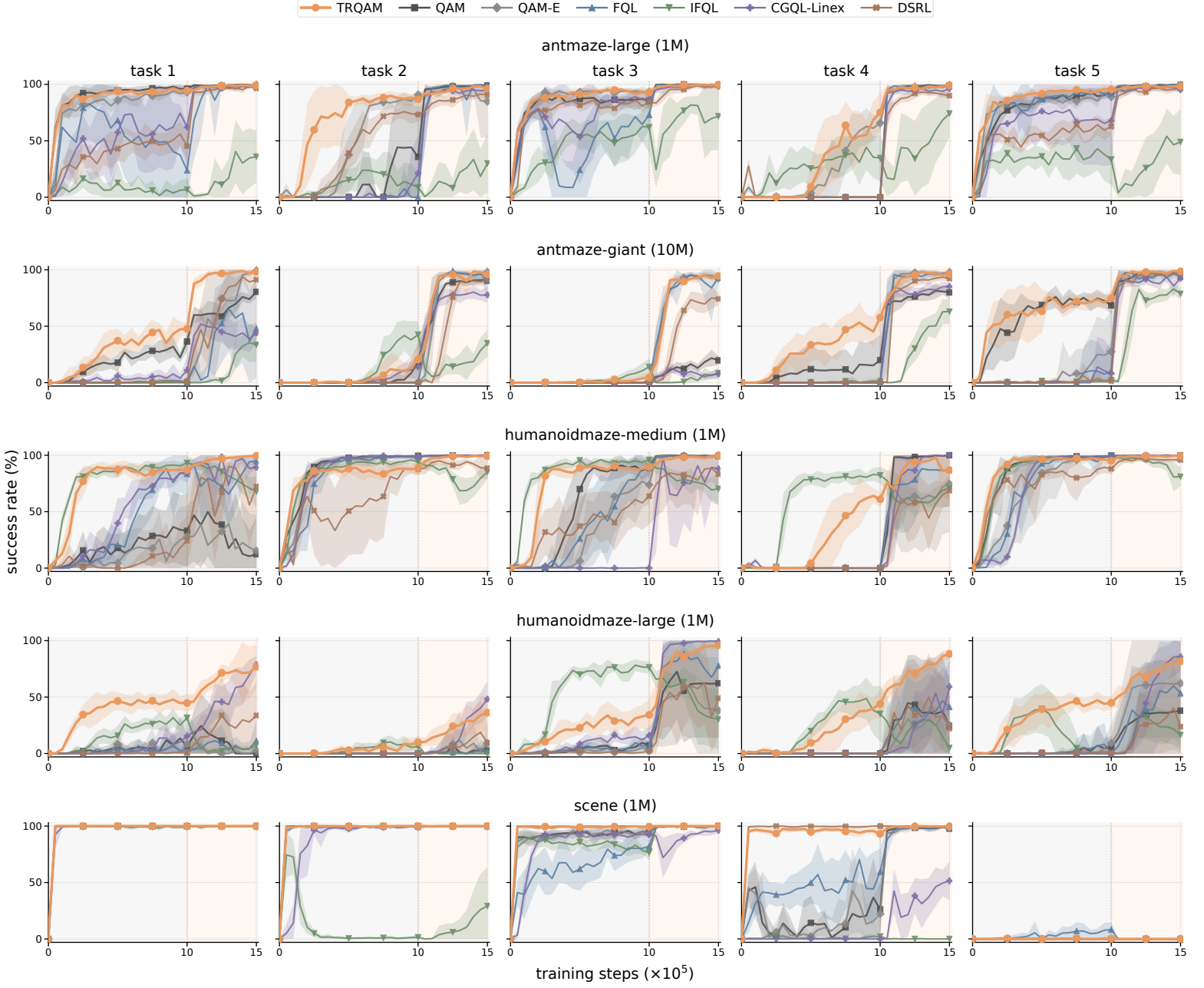


Figure 17: **Per-task offline-to-online learning curves on OGBench [38] (8 seeds)** Suites: antmaze-large, antmaze-giant, humanoidmaze-medium, humanoidmaze-large, scene. Each panel reports success rate over training steps for all seven methods (TRQAM, QAM, QAM-E, FQL, IFQL, CGQL-Linex, DSRL); the offline-to-online transition occurs at 10^6 steps. Shaded regions denote ± 1 standard deviation across seeds.

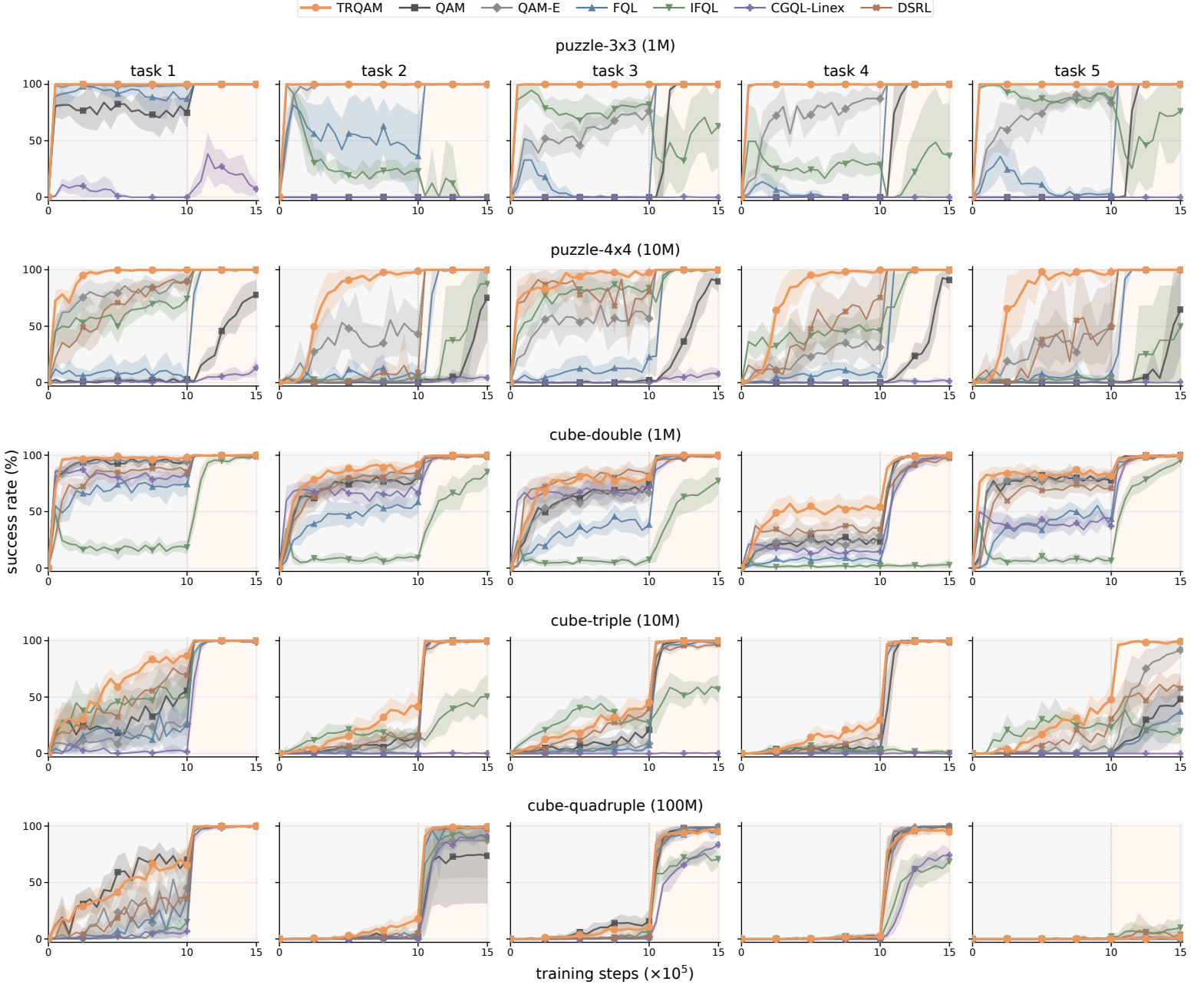


Figure 18: **Per-task offline-to-online learning curves on OGBench [38] (8 seeds)** Suites: puzzle-3x3, puzzle-4x4, cube-double, cube-triple, cube-quadruple. Each panel reports success rate over training steps for all seven methods (TRQAM, QAM, QAM-E, FQL, IFQL, CGQL-Linex, DSRL); the offline-to-online transition occurs at 10^6 steps. Shaded regions denote ± 1 standard deviation across seeds.